SUMO RL Traffic Light Control

Methodology & Research Findings

Comprehensive Documentation for Paper Writing

Tartalomjegyzék

I. SYSTEM ARCHITECTURE — A RENDSZER FELÉPÍTÉSE **4**

1.1 Multi-Agent RL Setup . . . 4

1.2 Observation Space — MIT LÁT AZ ÁGENS? . . . 5

1.3 Action Space — MIT CSINÁLHAT AZ ÁGENS? . . . 5

1.4 Step Mechanics — EGY LÉPÉS RÉSZLETEI . . . 5

II. TRAFFIC GENERATION — HOGYAN GENERÁLJUK A FORGALMAT? **6**

2.1 Lane-Level Random Traffic . . . 6

2.2 Episode Duration Rotation — VÁLTOZÓ SZIMULÁCIÓS HOSSZ . . . 8

2.3 Warm-up — KIKAPCSOLVA . . . 8

III. METRIC COLLECTION & SUMO API — HONNAN JÖNNEK A MÉRŐSZÁMOK? **8**

3.1 Raw SUMO Metrics — RÉSZLETES LEÍRÁS . . . 8

3.2 Sentinel Value Problem — A TRAVELTIME CSAPDÁJA . . . 9

3.3 Metric Units Summary — MÉRTÉKEGYSÉGEK ÖSSZEFOGLALÁSA . . . 10

IV. PCA ANALYSIS — MIÉRT PONT EZEKET A METRIKÁKAT HASZNÁLJUK? **10**

4.1 Purpose — KONKRÉT CÉL . . . 10

4.2 Data Collection — ADATGYŰJTÉS . . . 11

4.3 Preprocessing — ELŐFELDOLGOZÁS . . . 11

4.4 Results — Explained Variance . . . 11

4.5 Results — PCA Loadings (Component Weights) . . . 11

4.6 Interpretation . . . 12

4.7 PCA-Based Reward Metric Selection . . . 13

V. NORMALIZATION METHOD COMPARISON — MELYIK MÓDSZERREL NORMALIZÁLJUNK? **13**

5.1 The Problem — SZÁMOKKAL . . . 14

5.2 Methods Compared (10 methods) . . . 14

5.3 Quality Metric . . . 14
5.4 Results — Normalization Quality Ranking . . . 14
5.5 Why Not Rank Gauss for Online RL? . . . 15

VI. REWARD FUNCTION — A JUTALOM FÜGGVÉNY MATEMATIKÁJA 15
6.1 Log-Sigmoid Normalization Formula — LÉPÉSRŐL LÉPÉSRE . . . 15
6.2 Calibration Parameters — A KONSTANSOK ÉS HONNAN JÖNNEK . . . 16
6.3 Combined Reward — A KÉT METRIKA ÖSSZEVONÁSA . . . 16
6.4 Reward Function Properties . . . 17
6.5 Measurement Units Verification — MÉRTÉKEGYSÉG-ELLENŐRZÉS . . . 17

VII. NEURAL NETWORK ARCHITECTURE — A HÁLÓ FELÉPÍTÉSE 18
7.1 Algorithm: QR-DQN (Quantile Regression DQN) . . . 18
7.2 Policy Network — A HÁLÓ STRUKTÚRÁJA . . . 18
7.3 Model Saving — MENTÉS ÉS TRANSFER LEARNING . . . 19

VIII. HYPERPARAMETER SWEEP — HOGYAN KERESSÜK A LEGJOBB BEÁLLÍTÁST? 19
8.1 Sweep Method: Bayesian Optimization . . . 19
8.2 Swept Parameters — MIT KERESÜNK? . . . 19
8.3 Fixed Parameters — EZEKET NEM KERESSÜK . . . 20
8.4 Per-Episode Randomization — MINDEN EPIZÓDBAN MÁS . . . 20

IX. SIMULATION BACKEND — LIBSUMO vs TRACI 21
9.1 Two SUMO Connection Methods . . . 21
9.2 Selection Logic — AUTOMATIKUS VÁLASZTÁS . . . 21
9.3 SUMO Subscriptions — BATCH DATA RETRIEVAL . . . 22

X. CORRELATION MATRIX — MIT MUTAT A KORRELÁCIÓS MÁTRIX? 22
10.1 Strong Positive Correlations (>0.8) — EGYÜTT MOZGÓK . . . 22
10.2 Negative Correlations — ELLENTÉTES MOZGÁS . . . 22
10.3 Near-Zero Correlations — FÜGGETLEN METRIKÁK . . . 23
10.4 Interpretation for Reward Design — MIT JELENT EZ NEKÜNK? . . . 23

XI. DATA STATISTICS — REFERENCE VALUES 23
11.1 Per-Metric Distributions (527,747 samples, flow 100-1200 veh/h/lane) . . . 23

XII. LOG-SIGMOID NORMALIZATION PARAMETERS — ALL METRICS 24

XIII. ROBUST IQR NORMALIZATION PARAMETERS (alternative method) 25

XIV. GENERATED ANALYSIS CHARTS — DETAILED INTERPRETATION 25

XV. BASELINE EVALUATION — ÖSSZEHASONLÍTÓ STRATÉGIÁK 29
15.1 Stratégiák Összefoglalása . . . 29
15.2 Static Baseline . . . 30
15.3 Actuated Baseline . . . 30
15.4 Random Baseline . . . 30
15.5 Max-Pressure Baseline (irodalmi referencia) . . . 30
15.6 WandB Integráció . . . 31
15.7 Futtatás . . . 31

XVI. TRAINING PIPELINE — GUI ÉS CLI HIPERPARAMÉTER ÁTADÁS 31
16.1 GUI → Subprocess Hiperparaméter Átadás . . . 32
16.2 WandB Sweep Integráció . . . 32
16.3 Docker Pipeline . . . 32
16.4 Episode Logging . . . 32

XVII. CURRICULUM LEARNING — FOKOZATOS FORGALOMNÖVELÉS 33
17.1 Potenciális Előnyök . . . 33
17.2 Problémák a Megközelítéssel . . . 33
17.3 A Jelenlegi Megoldás: Domain Randomization + Target+Spread . . . 33
17.4 Ha Mégis Kipróbálnánk Curriculum-ot . . . 34

XVIII. KEY CONCLUSIONS FOR THE PAPER 34

XX. ARCHITEKTURÁLIS VÁLTÁS — JUNCTION-SZINTŰ INDEPENDENS TRÉNING 36

XIX. BIBLIOGRAPHY / REFERENCES 40

END OF DOCUMENT 41

==========
SUMO RL TRAFFIC LIGHT CONTROL — METHODOLOGY & RESEARCH FINDINGS Comprehensive Documentation for Paper Writing ==========
Generated: 2026-03-14 Data source: metric_pca_test_v2/ (527,747 samples) Simulation: SUMO (Simulation of Urban MObility) + traci/libsumo RL framework: Stable-Baselines3 QR-DQN (Quantile Regression DQN) Network: mega_catalogue_v2.net.xml — 21 signalized junctions (4×5 + 2 split) ==========

EZ A DOKUMENTUM CÉLJA: Mindent tartalmaz, amit a cikk megírásához tudni kell:

- Mi a rendszer és hogyan működik (I-II.)
- Honnan jönnek a mérőszámok és mit jelentenek (III.)
- Miért pont ezeket a metrikákat választottuk a reward-hoz — PCA elemzés (IV.)
- Miért pont a Log-Sigmoid normalizációt használjuk (V.)
- A reward függvény matematikai leírása, számokkal (VI.)
- A neurális háló és a hiperparaméter keresés (VII-VIII.)
- Minden ábra részletes értelmezése (XIV.)
- Training pipeline: GUI, CLI, Docker (XVI.)
- Curriculum learning vs domain randomization (XVII.)

Ha valami nem világos, az ábrák a metric_pca_test_v2/ mappában vannak. A kód: sumo_rl_environment.py (RL env), rl_trainer.py (tanítás), metric_collection_test.py (PCA adatgyűjtés), normalization_comparison.py.

FONTOS: XX. fejezetben leírt ARCHITEKTURÁLIS VÁLTÁS: A rendszer átállt junction-szintű independens tréningre. Többé NEM egyszerre tanul 21 junction, hanem egyenként, saját reward kalibrációval. A részleteket és implementációs checklistet lásd a XX. fejezetben.

I. SYSTEM ARCHITECTURE — A RENDSZER FELÉPÍTÉSE

A NAGY KÉP: Van egy városi úthálózatunk (SUMO szimulátorban), 21 jelzőlámpás csomóponttal. Mindegyik csomóponthoz tartozik egy "ágens" — egy mesterséges intelligencia, amely megtanulja, mikor melyik fázist (zöld irányt) kapcsolja be. Az ágensek egymástól függetlenül tanulnak (nincs kommunikáció köztük), de ugyanazt a hálózati architektúrát használják, külön-külön súlyokkal.

A tanulás lényege: az ágens cselekszik (fázist vált), megfigyeli mi történik (forgalmi metrikák), és kap egy jutalmat (reward). A jutalom alapján megtanulja, hogy melyik helyzetben melyik fázis a legjobb.

1.1 Multi-Agent RL Setup

- 21 csomópont, mindegyik önálló RL ágens
- Csomópont azonosítók: R1C1_C ... R4C5_C (4×5 rács), R4C2_CE/R4C2_CW (kettéosztott)
- Algoritmus: QR-DQN (Quantile Regression Deep Q-Network)

MIÉRT QR-DQN ÉS NEM SIMA DQN? A hagyományos DQN egyetlen számot (átlagos várható érték) becsül minden cselekvésre. A QR-DQN viszont az EGÉSZ ELOSZLÁST becsli — tehát tudja, hogy "ez a cselekvés átlagosan jó, de nagy a variancia". **Ez fontos, mert a forgalom természeténél fogva SZTOCHASZTIKUS:** ugyanaz a jelzésprogramozás néha jól működik, néha nem, a véletlenszerű forgalmi minták miatt. A QR-DQN ezt a bizonytalanságot kezeli.

MIÉRT FÜGGETLEN TANULÁS (NEM KOOPERATÍV)? Kooperatív tanulás (ahol az ágensek kommunikálnak) exponenciálisan növekvő állapotteret jelent 21 ágensre. A független tanulás skálázhatóbb, és a gyakorlatban jól működik, mert a szomszédos csomópontok hatása az observation-ön keresztül implicit megjelenik (a detektor adatok tükrözik a szomszéd forgalmát).

1.2 Observation Space — MIT LÁT AZ ÁGENS?

Minden lépésben az ágens a következő információkat kapja:

"phase" (1 szám): Melyik zöld fázis van éppen bekapcsolva (0, 1, 2, ...). Pl. 0 = észak-dél zöld, 1 = kelet-nyugat zöld.

"phase_timer" (1 szám, 0.0 és 1.0 között): Mennyi ideje tart az aktuális fázis, normalizálva. 0.0 = éppen váltott, 1.0 = már 600 másodperce fut (10 perc!). Képlet: min(phase_timer / 120.0, 1.0), ahol 120 lépés = 600 sec. Ez azért fontos, mert az ágens tudja, hogy "már régóta zöld van ezen az irányon, lehet, hogy érdemes váltani".

"occupancy" (annyi szám, ahány detektor van): A detektorok foglaltsága 0 és 1 között. 0 = üres (nincs jármű a detektor felett), 1 = folyamatosan foglalt. Ez a legfontosabb bemeneti adat — ebből "érzi" az ágens a forgalmat.

"flow" (annyi szám, ahány detektor van): Hány jármű haladt át a detektoron a lépés alatt (átlag/substep).

FONTOS: a detektorok induktív hurokdetektorok (induction loops), amelyek a SUMO hálózatban definiálva vannak (detectors.add.xml). Minden bejövő sávon van legalább egy detektor, jellemzően a jelzőlámpa előtt.

1.3 Action Space — MIT CSINÁLHAT AZ ÁGENS?

Discrete(num_phases) — annyi lehetséges cselekvés, ahány zöld fázis van.

Pl. ha 4 fázis van: {0, 1, 2, 3} 0 = Észak-Dél egyenes + jobbra kanyar 1 = Észak-Dél balra kanyar 2 = Kelet-Nyugat egyenes + jobbra kanyar 3 = Kelet-Nyugat balra kanyar

AZ INTERGREEN RENDSZER: Ha az ágens fázist akar váltani (pl. 0→2), NEM ugorhat azonnal! A valóságban is van egy "köztes idő" (intergreen time), ami alatt: 1. Az aktuális fázis sárgára vált (figyelmeztetés) 2. Egy rövid ideig minden piros (biztonság — senki se menjen!) 3. Az új fázis zöldre vált

Ez az intergreen mátrixból jön — minden fázis-párra definiálja a szükséges átmeneti fázisokat és azok időtartamát. Ez a valóságot szimulálja: egy jelzőlámpa nem válthat piros-zöld pillanat alatt.

MINIMUM ZÖLD IDŐ: Az ágens nem válthat zöldet azonnal az előző váltás után. min_green_time = 5 sec → legalább 5 másodpercig kell zöldnek lennie. Ez megakadályozza a "villogást" (ami a valóságban tilos és veszélyes).

1.4 Step Mechanics — EGY LÉPÉS RÉSZLETEI

delta_time = 5 SUMO másodperc / RL lépés

Ez azt jelenti: az ágens NEM minden szimulációs másodpercben dönt, hanem 5 másodpercenként. Ez reális — a valóságban sem változtatjuk a jelzésprogramot másodpercenként.

Egy lépés belső folyamata: 1. Az ágens kap egy observation-t és választ egy akciót (target phase). **2. 5× SUMO substep (5 szimuált másodperc):** Minden substepben: a) update_logic(): a jelzőlámpa logika frissítése

- Ha éppen fázisváltás történik: a transition_queue-ból a következő köztes fázis beállítása (sárga, piros, stb.)
- Ha nem: min_green_timer csökkentése b) traci.simulationStep(): a SUMO 1 másodpercet szimulál
- Járművek mozognak, sávot váltanak, megállnak, stb. c) collect_measurements(): a metrikák gyűjtése
- Minden bejövő sáv várakozási ideje, CO2, sebesség, stb.
- Detektor foglaltság és átfolyás 3. Observation számítás: a 5 substep alatt akkumulált értékek átlagolása (osztás steps_measured-del). 4. Reward számítás: Log-Sigmoid normalizáció (lásd VI. fejezet). 5. Done ellenőrzés: ha a szimulációs idő >= traffic_duration → vége.

II. TRAFFIC GENERATION — HOGYAN GENERÁLJUK A FORGALMAT?

A PROBLÉMA: Az RL ágensnek forgalomra van szüksége, hogy tanulhasson. De nem egy fix forgalmat akarunk, hanem VÁLTOZATOSAT — különböző forgalmi szinteket, véletlenszerű mintákat, különböző időtartamokat. Így az ágens nem tanul rá egy konkrét szituációra, hanem ÁLTALÁNOSÍT.

2.1 Lane-Level Random Traffic

Metódus: generate_random_traffic(flow_range=(100, 900)) Fájl: sumo_rl_environment.py, generate_random_traffic() függvény

MIÉRT NEM A SUMO BEÉPÍTETT randomTrips.py-t HASZNÁLJUK? A randomTrips.py O-D (Origin-Destination) utazásokat generál a teljes hálózatra — de így nem tudjuk kontrollálni, hogy egy adott SÁVBA hány jármű érkezzen. A forgalommérnöki gyakorlatban a forgalmi terhelést SÁV SZINTEN definiáljuk (pl. "ezen a sávon 500 jármű/óra"). Ezt a randomTrips nem tudja.

AZ ALGORITMUS LÉPÉSRŐL LÉPÉSRE: 1. A sumolib könyvtárral beolvassuk a hálózatot (mega_catalogue_v2.net.xml) 2. Minden vezérelt csomóponthoz megkeressük a BEJÖVŐ SÁVOKAT. **Pl. R1C1_C csomóponthoz:** az északi, déli, keleti, nyugati irányból érkező sávok (jellemzően 6-12 sáv csomópontonként). 3. Minden sávhoz meghatározzuk a lehetséges CÉLPONTOKAT. A SUMO hálózatban a sávkapcsolatok (connections) definiálják, hogy melyik sávról melyik kimenő élre lehet eljutni. **4. MINDEN SÁVRA véletlenszerű forgalmi szintet generálunk:** flow_per_hour = random(flow_min, flow_max) — pl. random(100, 900) 5. A járműveket egyenletes időközökkel indítjuk, ±50% véletlenszerű eltolással (jitter), hogy ne legyen tökéletesen metronomikus. 6. Minden járműhöz:

- departLane attribútum → a jármű a megadott sávba lép be
- Véletlenszerű célpont a sáv elérhető céljai közül 7. Időrend szerinti rendezés, XML route fájl kiírása.

FORGALMI TARTOMÁNY — EPIZÓD-SZINTŰ TARGET + SPREAD RANDOMIZÁLÁS A korábbi megoldás (flow_range=(100, 900) uniform per lane) problémája: ha sok sáv van (~80), a nagy számok törvénye miatt az ÖSSZ-forgalom mindig ~500 veh/h/lane körül áll be — az ágens nem lát igazán könnyű vagy nehéz epizódokat.

Az ÚJ megoldás két szinten randomizál: 1. target = random.choice([100, 200, 300, 400, 500, 600, 700, 800, 900]) → az epizód összforgalmának nagyságrendje 2. spread = random.choice([50, 100, 150, 200, 250, 300]) → az irányok közötti aszimmetria mértéke 3. flow_range = (max(50, target-spread), min(1100, target+spread)) 4. Sávonként: flow = randint(flow_min, flow_max)

Példák: target=200, spread=50 → [150, 250] — könnyű, egyenletes forgalom target=200, spread=300 → [50, 500] — könnyű átlag, nagy aszimmetria target=700, spread=50 → [650, 750] — nehéz, egyenletes target=700, spread=300 → [400, 1000] — nehéz átlag, nagy aszimmetria target=500, spread=300 → [200, 800] — hasonló a régi [100,900]-hoz

Ezzel az ágens lát:

- Könnyű ÉS nehéz epizódokat (target variál)
- Egyenletes ÉS aszimmetrikus forgalmat (spread variál)
- Abszolút min: 50, max: 1100 veh/h/lane (gridlock elkerülése)

Ha a baseline_actuated.py vagy bármely külső hívó explicit flow_range-t ad az options-ben, az felülírja a target+spread logikát.

FORGALMI SZINTEK ÉRTELMEZÉSE: 100 veh/h/lane ≈ 36 másodpercenként jön egy autó → SZABAD FORGALOM Ilyenkor alig van sor, szinte minden autó zöldet kap. A reward magas lesz (~0.8), és az ágens nem kap sok "tanulnivalót".

400-600 veh/h/lane ≈ 6-9 másodpercenként egy autó → TIPIKUS VÁROSI A legtöbb városi főútvonal ebben a tartományban van. A jelzőlámpa programozása SZÁMÍT — rossz időzítés = sor, jó = áramlás.

900 veh/h/lane ≈ 4 másodpercenként egy autó → TELÍTŐDÉS KÖZELÉBEN Egy sáv kapacitása jelzőlámpánál ~1800 veh/h (zöld időarány ≈50%). 900 veh/h tehát ~50%-os kihasználtság — már komoly sorok képződhetnek.

1100 veh/h/lane ≈ 3.3 másodpercenként → MAXIMÁLIS TRÉNING SZINT A target+spread logika legfeljebb 1100-ig engedi. 1200+ a PCA elemzésben volt, de a tanításban nem — gridlock kockázat.

PCA KALIBRÁCIÓ — HÁLÓZAT-FÜGGŐ NORMALIZÁCIÓ: A reward normalizációs konstansok HÁLÓZAT-FÜGGŐEK, mert a forgalmi metrikák skálája erősen függ a hálózat méretétől és topológiájától.

MEGA CATALOGUE (21 junction, 527K+ minta, metric_pca_test_v2/): MU_WAIT=4.5848, STD_WAIT=1.8249 (medián wait: 120.2 sec) **MU_CO2=10.8706, STD_CO2=0.9629 (medián CO2:** 70,129 mg/s)

OSM SINGLE JUNCTION (2632893078, 63K minta, metric_pca_osm/): MU_WAIT=2.4486, STD_WAIT=1.8586 (medián wait: 14.6 sec) **MU_CO2=9.9652, STD_CO2=0.7381 (medián CO2:** 23,246 mg/s)

ELTÉRÉS: MU_WAIT -2.14, MU_CO2 -0.91 → az OSM junction lényegesen kisebb forgalmi terhelést generál. Ha a mega konstansokat használjuk az OSM junction-re, a sigmoid szinte minden értéket "jónak" lát (reward átlag 0.69, szűk range 0.35) → az agens nem tud tanulni, mert nincs érdemi reward grádiens a jó és rossz akciók között.

MEGOLDÁS: sumo_rl_environment.py a single_agent_id alapján választ konstansokat — single agent → OSM, multi agent → mega catalogue. Új hálózatoknál a metric_collection_osm.py mintájára kell kalibrálni.

JÁRMŰTÍPUS PARAMÉTEREK (realisztikus személygépjármű): accel = 0.8 m/s² (gyorsulás — átlagos személyautó) decel = 4.5 m/s² (fékezés — komfortos fékezés) sigma = 0.5 (Krauss modell noise — mennyi a "vigyázatlanság") length = 5m (átlag autóhossz) minGap = 2.5m (minimális követési távolság álló helyzetben) maxSpeed = 16.67 m/s (60 km/h — tipikus városi sebességhatár)

FONTOS: Minden epizód elején (reset()) ÚJ véletlenszerű forgalmat generálunk. Tehát soha nem ugyanaz a forgalmi minta — az ágensnek alkalmazkodnia kell. Ez a kulcsa az általánosításnak.

2.2 Episode Duration Rotation — VÁLTOZÓ SZIMULÁCIÓS HOSSZ

Minden epizódban véletlenszerűen kiválasztunk egy időtartamot: [900, 1800, 2700, 3600, 5400, 7200] másodperc [15 perc, 30 perc, 45 perc, 1 óra, 1.5 óra, 2 óra]

MIÉRT ROTÁLUNK?

- A valóságban a csúcsidőszakok különböző hosszúak. Reggeli csúcs: 1-1.5 óra. Esti csúcs: 1.5-2 óra. Hétvégi vásárlás: 30-45 perc.
- Ha mindig 1 órát szimulálnánk, az ágens MEGTANULNÁ, hogy "a 720. lépés után vége van" → nem általánosítana.
- Rövid epizódok (15-30 perc): gyorsabb feedback, de zajosabb metrikák (kevesebb adat az átlagoláshoz).
- Hosszú epizódok (1.5-2 óra): stabilabb átlagok, de lassabb tanulás.
- A keverés MEGAKADÁLYOZZA a temporális overfitting-et.

Felülírható: options={'traffic_duration': 3600} a reset()-ben.

2.3 Warm-up — KIKAPCSOLVA

Korábbi megoldás: a reset() elején 50-150 random másodperc warm-up, ahol a jelzőlámpák VÉLETLENSZERŰ fázisokat kaptak. Ez dugót okozott, amit az ágens a legelső lépésben megörökölt.

MIÉRT LETT KIKAPCSOLVA? 1. A random fázisok kaotikus jelzésprogramot adtak → dugó képződött 2. Az ágens memória nélküli (nincs LSTM/Transformer), tehát nem tudja megkülönböztetni a "megörökölt dugó" és a "saját maga által okozott dugó" szituációkat → zajos, nem informatív tanulási jel 3. A forgalom a jitter miatt úgyis fokozatosan indul (depart $\approx 0 +$ jitter), tehát az első 1-2 másodpercben már vannak járművek a hálózatban 4. A detektorok az első RL lépésnél (5 sec) már látnak forgalmat

Felülírható: options={'warmup_seconds': 60} a reset()-ben (pl. baseline teszthez).

III. METRIC COLLECTION & SUMO API — HONNAN JÖNNEK A MÉRŐSZÁMOK?

A LÉNYEG: Minden RL lépésben (5 szimuált másodperc) a SUMO szimulátorból kiolvasunk egy csomó adatot. Ezeket az adatokat AKKUMULÁLJUK (összegezzük), majd az 5 substep átlagát vesszük. Ebből lesz a reward kiszámítható.

A kód: sumo_rl_environment.py → TrafficAgent.collect_measurements()

GYŰJTÉSI MECHANIZMUS: Az environment reset_step_metrics()-t hív minden RL lépés elején. Ez NULLÁZZA az akkumulátorokat (accumulated_waiting_time, stb.). Aztán 5× lefut a collect_measurements(), ami HOZZÁADJA az aktuális értékeket. A végén osztunk a steps_measured-del (=5). Tehát ami a reward-ba megy, az egy 5 másodperces ablak átlaga.

3.1 Raw SUMO Metrics — RÉSZLETES LEÍRÁS

1. TotalWaitingTime [sec] **SUMO hívás:** traci.lane.getWaitingTime(lane) **MIT AD VISSZA:** Az adott sávon MOST várakozó összes jármű összesített várakozási ideje, másodpercben. "Várakozás" = a jármű sebessége < 0.1 m/s (gyakorlatilag áll). **HOGYAN GYŰJTJÜK:** Összegezzük az összes bejövő sáv értékét, majd az 5 substep átlagát vesszük. **MIÉRT EZ A FŐ REWARD**

METRIKA: Közvetlenül méri, amit javítani akarunk — mennyi időt töltenek az autók piros lámpánál várakozva. A jelzőlámpa programozása KÖZVETLENÜL befolyásolja ezt az értéket.

2. AvgWaitingTime [sec/jármű] = TotalWaitingTime / VehCount Nem a reward-ban, csak az elemzésben (PCA) használjuk. Azért nem ezt használjuk reward-nak, mert kis forgalomnál (1-2 autó) egy rosszul járó autó extrém magas átlagot ad.

3. TotalCO2 [mg/s] **SUMO hívás:** traci.lane.getCO2Emission(lane) **MIT AD VISSZA:** Az adott sáv összes járművének pillanatnyi CO2 kibocsátási rátája, milligramm/másodpercben. A HBEFA (Handbook Emission Factors for Road Transport) modellt használja — európai emissziós szabvány. A kibocsátás függ a jármű sebességétől és gyorsulásától: álló motor → alacsony CO2, gyorsulás → magas CO2, állandó sebesség → közepes CO2. **HOGYAN GYŰJTJÜK:** Összegezzük az összes bejövő sáv értékét. **MIÉRT VAN A REWARD-BAN:** Környezeti szempont. Nem teljesen redundáns a várakozási idővel — a stop-and-go forgalom több CO2-t termel, mint a lassú de egyenletes haladás.

4. AvgCO2 [mg/s/jármű] = TotalCO2 / VehCount EZ SZINTE KONSTANS (~1575 mg/s/jármű)! Miért? Mert alacsony városi sebességeknél a HBEFA modell kibocsátása nagyrészt az alapjárat + lassú haladás emissziója, ami nem változik sokat. Az autók kb. ugyanannyit bocsátanak ki akár állnak, akár lassan haladnak. **KÖVETKEZMÉNY:** Nem alkalmas reward metrikának! Nincs benne információ, amiből az ágens tanulhatna. (A PCA ezt igazolta.)

5. TotalTravelTime [sec] — SZŰRT VERZIÓ **SUMO hívás:** traci.lane.getTraveltime(lane) **MIT AD VISSZA:** Mennyi idő alatt lehet áthaladni a sávon az aktuális forgalmi viszonyok mellett. **VIGYÁZAT — SENTINEL ÉRTÉKEK:** Ha a sáv üres vagy teljesen bedugult, a SUMO ~200,000 és ~5,000,000 közötti értékeket ad vissza. Ez NEM valós menetidő, hanem a SUMO implementációs "jelzőértéke" (sentinel value). Lásd: 3.2. **SZŰRÉS:** Csak $0 < tt < 1000$ értékeket akkumulálunk. 1000 sec reális maximum egy ~90 méteres sávra.

6. AvgTravelTime [sec] = TotalTravelTime / valid_tt_lanes (szűrt) Elemzési metrika, nem reward.

7-10. VehCount, AvgSpeed, AvgOccupancy, QueueLength Szintén elemzési metrikák. Részletek a korábbi verzióban.

3.2 Sentinel Value Problem — A TRAVELTIME CSAPDÁJA

Ez egy NAGYON fontos felismerés, ami a PCA elemzésből derült ki!

A SUMO traci.lane.getTraveltime(lane) függvénye:

- Ha vannak járművek a sávon: normális érték (10-1000 sec)
- **Ha NINCS jármű:** ~200,000 - ~5,000,000 értéket ad vissza

Ez egy SUMO implementációs artifact — a sáv hosszát elosztja egy nagyon kicsi sebességgel (ami üres sávnál 0.0001 m/s körüli).

Ha ezt NEM szűrjük ki, a TravelTime teljesen használhatatlan:

- Az átlag 877,339 sec lesz (a valóság ~725 sec)
- A szórás 2,175,933 (!!!)
- Az eloszlás extrém jobbra ferdült

A PCA IGAZOLTA a szűrés szükségességét:

- Raw TT (szűretlen) → PC3-ra kerül (12.3%) = ZAJDIMENZIÓ
- Filtered TT (szűrt) → PC2-ra kerül (22.6%) = VALÓS DIMENZIÓ Ez azt jelenti, hogy a raw TT egy teljesen más típusú variabilitást mér (sentinel noise), nem menetidőt.

TANULSÁG: A cikkben érdemes megemlíteni, hogy a sentinel értékek kezelése kritikus — a PCA

objektíven igazolta, hogy a szűretlen menetidő zajt hoz be, nem információt.

3.3 Metric Units Summary — MÉRTÉKEGYSÉGEK ÖSSZEFOGLALÁSA

Metrika | Mértékegység | SUMO forrás | Reward?

```
--------------------|-------------|--------------------------------|---------
TotalWaitingTime    | sec/step    | lane.getWaitingTime()          | IGEN
AvgWaitingTime      | sec/jármű   | TotalWaiting / VehCount        | nem
TotalCO2            | mg/s/step   | lane.getCO2Emission()          | IGEN
AvgCO2              | mg/s/jármű  | TotalCO2 / VehCount            | nem (konstans!)
TotalTravelTime     | sec/step    | lane.getTraveltime() szűrt     | nem
AvgTravelTime       | sec/jármű   | TotalTT / valid_lanes          | nem
VehCount            | járművek    | lane.getLastStepVehicleNumber() | nem
AvgSpeed            | m/s         | detektor adatokból             | nem
AvgOccupancy        | %           | inductionloop.getLastStepOcc() | nem
QueueLength         | járművek    | megállt járművek száma         | nem

 = a reward függvényben használt metrikák (lásd VI. fejezet)
```

IV. PCA ANALYSIS — MIÉRT PONT EZEKET A METRIKÁKAT HASZNÁLJUK?

A NAGY KÉRDÉS: Van 12 forgalmi metrikánk (WaitingTime, CO2, TravelTime, Speed, stb.). Mindegyiket használhatnánk reward-ként. De melyiket ÉRDEMES?

A NAIV MEGKÖZELÍTÉS ROSSZ LENNE: Ha mind a 12-t beraknánk a reward-ba, az (1) lassítaná a tanulást, (2) egymásnak ellentmondó gradieseket adna, és (3) a redundáns metrikák (amelyek ugyanazt mérik) aránytalan súlyt kapnának.

A PCA (Principal Component Analysis = Főkomponens Elemzés) megoldja: Megmondja, hogy a 12 metrika VALÓJÁBAN hány FÜGGETLEN információ- dimenziót tartalmaz, és melyik metrika melyik dimenzióhoz tartozik.

EGYSZERŰ ANALÓGIA: Képzeld el, hogy 12 hőmérőd van egy szobában. 6 a szoba közepén, 3 az ablak mellett, 3 a radiátor mellett. A 6 középső UGYANAZT méri **(szobahőmérséklet), csak minimális eltéréssel. A PCA megmondaná:** "Van 3 független dimenzió: szobahő, ablakhő, radiátorhő. A 6 középső hőmérő redundáns — elég egy belőlük."

4.1 Purpose — KONKRÉT CÉL

Tudományosan igazolni, hogy: 1. A TotalWaitingTime a legjobb reward metrika (PC1 domináns) 2. A TotalCO2 hasznos kiegészítő (PC1, de más aspektus) 3. Az AvgCO2 HASZONTALAN (közel konstans, nulla PCA loading) 4. A raw TravelTime ZAJT hoz be (PC3 = sentinel noise) 5. A QueueLength, VehCount, Occupancy REDUNDÁNSAK a WaitingTime-mal

4.2 Data Collection — ADATGYŰJTÉS

HONNAN JÖNNEK AZ ADATOK? Ugyanabból a szimulációból, mint ami az RL tanításban fut! A metric_collection_test.py futtatta a SUMO-t 12 forgalmi szinten (100, 200, 300, ..., 1200 veh/h/lane), mindegyiket 3×, és minden lépésben kimentette mind a 12 metrikát 21 csomópontra.

MENNYISÉG: 12 forgalmi szint × 3 epizód × 21 csomópont × ~720 lépés/epizód = 527,747 érvényes adatpont (VehCount > 0 szűrés után)

FONTOS — KONZISZTENCIA AZ RL-LEL: Az adatok PONTOSAN úgy vannak kiszámítva, mint a reward függvényben:

- Akkumulált átlag a delta_time substepekre
- Ugyanazok a SUMO API hívások
- Ugyanaz a szűrési logika (TT < 1000) Ha más módszerrel gyűjtenénk az adatokat, a PCA paraméterek NEM lennének alkalmazhatók a reward-ban! Ez volt a legfontosabb döntés.

4.3 Preprocessing — ELŐFELDOLGOZÁS

A PCA érzékeny a skálázásra, ezért előkészítés szükséges:

1. SZŰRÉS: Csak azok a sorok, ahol VehCount > 0 Üres csomópontok (nincs jármű) torzítanák az elemzést.

2. LOG TRANSZFORMÁCIÓ: log(x + 0.00001) minden metrikára MIÉRT? A forgalmi metrikák JOBBRA FERDÜLTEK — néhány extrém nagy érték (pl. WaitingTime 31,000 sec) eltorzítja az átlagot. A log(x) "összenyomja" a nagy értékeket: log(10) = 2.3, log(100) = 4.6, log(1000) = 6.9, log(10000) = 9.2 Így a 100× különbség csak 4× különbség lesz log skálán. **FONTOS:** Ugyanezt a log transzformációt használjuk a reward-ban is!

3. STANDARDIZÁLÁS: (x - átlag) / szórás minden metrikára Hogy minden metrika egyforma "súlyú" legyen a PCA-ban. Anélkül a nagy értékű metrikák (TotalCO2 ~70,000) elnyomnák a kicsiket (AvgWaitingTime ~5).

4. PCA: Teljes felbontás 12 komponensre

4.4 Results — Explained Variance

Component | Explained Variance | Cumulative | Interpretation

```
----------|-------------------|------------|---------------------------
PC1       | 44.8%             | 44.8%      | Congestion dimension
PC2       | 22.6%             | 67.4%      | Travel Time dimension
PC3       | 12.3%             | 79.7%      | Raw TT sentinel noise
PC4       | 6.1%              | 85.8%      | Minor variation
PC5-12    | <5% each          | 100%       | Negligible

3 components capture 79.7% of total variance.
PC1+PC2 = 67.4% → two dimensions explain most of the real information.
```

4.5 Results — PCA Loadings (Component Weights)

Metric PC1 PC2 PC3 ——————————————————-

```
TotalWaitingTime        0.4001   0.0758   0.0449   ← PC1 dominant
AvgWaitingTime          0.3787   0.0327   0.1027   ← PC1 dominant
QueueLength             0.3837   0.1152   0.0707   ← PC1 dominant
VehCount                0.3694   0.1752  -0.1825   ← PC1 dominant
TotalCO2                0.3346   0.1714  -0.2798   ← PC1 dominant
AvgOccupancy            0.3089   0.1915  -0.2449   ← PC1 dominant
--------------------------------------------------
TotalTravelTime        -0.0483   0.5695   0.2386   ← PC2 dominant
AvgTravelTime          -0.0364   0.5634   0.2584   ← PC2 dominant
AvgSpeed               -0.2810   0.3322   0.3046   ← PC2 (inverse)
--------------------------------------------------
TotalTravelTime_Raw     0.2509  -0.2509   0.5162   ← PC3 = sentinel noise
AvgTravelTime_Raw       0.2445  -0.2602   0.5225   ← PC3 = sentinel noise
--------------------------------------------------
AvgCO2                 -0.0086   0.0222  -0.2342   ← NEAR ZERO on PC1,PC2
```

4.6 Interpretation

PC1 — CONGESTION DIMENSION (44.8%):

```
  TotalWaitingTime, AvgWaitingTime, QueueLength, VehCount, TotalCO2, AvgOccupancy
  all load strongly onto PC1 (0.31-0.40).

  These metrics are REDUNDANT: they all measure the same underlying phenomenon
  (traffic congestion). Using all of them in a reward function would be
  mathematically equivalent to using just one, multiplied by a constant.

  → Choose ONE from this cluster for the reward. We choose TotalWaitingTime
    because it has the highest PC1 loading (0.4001) and is directly
    actionable by traffic signal timing.

PC2 - TRAVEL TIME DIMENSION (22.6%):
  TotalTravelTime and AvgTravelTime load onto PC2 (~0.57).
  This represents INDEPENDENT information from congestion.

  However, travel time is LESS directly controllable by signal timing
  than waiting time (it includes free-flow travel time), and its
  filtered version discards sentinel values, reducing reliability.

  → Not included in reward. Monitored as evaluation metric.

PC3 - SENTINEL NOISE (12.3%):
  TotalTravelTime_Raw and AvgTravelTime_Raw load onto PC3 (~0.52).
  This dimension captures the SUMO sentinel value artifacts.

  → CONFIRMS that raw TT must be filtered. Never use raw TT in reward.

AvgCO2 - NEAR-ZERO LOADING ( constant):
  PC1 loading = -0.0086, PC2 loading = 0.0222
  AvgCO2  1575 mg/s/vehicle across all conditions.
  The per-vehicle CO2 rate is nearly constant because at low urban
```

```
  speeds, HBEFA emission is dominated by idle+cruise emissions.

  → AvgCO2 is USELESS as a reward metric. It provides no learning signal.
  → BUT TotalCO2 (loading 0.33 on PC1) IS useful because it scales
    with vehicle count - more vehicles = more total emissions.
```

4.7 PCA-Based Reward Metric Selection

SELECTED FOR REWARD: 1. TotalWaitingTime (PC1 loading: 0.40) — primary congestion signal **2. TotalCO2 (PC1 loading:** 0.33) — environmental dimension

WHY THESE TWO:

- Both from PC1 cluster (congestion), but TotalCO2 adds a secondary environmental optimization objective
- While correlated (both on PC1), their correlation is not perfect: TotalCO2 also depends on speed profile (acceleration/deceleration), which TotalWaitingTime doesn't capture
- Using 50/50 weighted combination: reward = 0.5×r_wait + 0.5×r_co2

WHY NOT OTHERS:

- **QueueLength, VehCount, AvgOccupancy:** redundant with TotalWaitingTime (PC1)
- TravelTime: less controllable, sentinel value issues
- AvgCO2: constant → zero gradient → no learning signal
- AvgSpeed: partially on PC2, but inversely → covered by TravelTime

V. NORMALIZATION METHOD COMPARISON — MELYIK MÓDSZERREL NORMALIZÁLJUNK?

A PROBLÉMA EGYSZERŰEN: Az RL ágensnek a reward-ot 0 és 1 között kell kapnia (vagy legalább egy kiszámítható, stabil tartományban). De a nyers forgalmi metrikák:

```
  TotalWaitingTime: 0.2-tól 31,348 sec-ig (150,000× különbség!)
  TotalCO2: 6.4-től 246,813 mg/s-ig (38,000× különbség!)
Ezeket valahogy "össze kell préselni" a [0, 1] tartományba.

DE NEM MINDEGY, HOGYAN!
Egy rossz normalizáció:
  - Az értékek 99%-át 0.01-0.02 közé nyomja → az ágens nem tud
    különbséget tenni "közepes" és "jó" között → nem tanul
  - Extremális értékeknél ugrásszerű változás → instabil tanulás
  - A forgalmi szint változásával a normalizált eloszlás "elmászik"

PLUSZ: ONLINE vs OFFLINE:
Offline módszerek (pl. Rank Gauss) a TELJES adathalmazt igénylik.
De az RL tanítás során az adatok EGYENKÉNT jönnek, valós időben.
Nincs "teljes adathalmaz" - amit eddig láttunk, az a tanítás
eddigi tapasztalata, ami folyamatosan bővül. Ezért ONLINE-
kompatibilis módszer kell: olyan, amit 2-4 előre kiszámított
```

```
konstanssal paraméterezünk, és utána semmit nem kell frissíteni.
```

5.1 The Problem — SZÁMOKKAL

RL reward must be in a bounded, well-behaved range [0, 1]. Raw traffic metrics span vastly different scales:

```
  TotalWaitingTime: 0.2 - 31,348 sec
  TotalCO2: 6.4 - 246,813 mg/s
  VehCount: 0 - hundreds

Additionally, for ONLINE RL training:
  - No full dataset available a priori
  - Parameters must be precomputed from representative data
  - Method must work with streaming data
```

5.2 Methods Compared (10 methods)

1. Raw Data — no normalization (baseline) 2. Tanh (Z-score) — tanh((x - mean) / std) 3. Algebraic — x / sqrt(1 + x^2) 4. Log-Normal — log(x + eps) 5. Log-Sigmoid — 1 - sigmoid((log(x) -) /) ← SELECTED 6. Robust (IQR) — tanh((x - median) / IQR) 7. Rank Gauss — rank transform → inverse normal CDF 8. Box-Cox — Box-Cox power transform 9. Clipped 95% — clip to [p5, p95], then linear scale to [0, 1] 10. Harmonic — K / (x + eps), where K = percentile(data, 5)

5.3 Quality Metric

quality = |skewness| + |excess_kurtosis|

Perfect Gaussian → quality = 0 Lower = better normalized (more symmetric, fewer outliers)

5.4 Results — Normalization Quality Ranking

Rank | Method | Avg Quality | Online? | Notes

```
-----|-----------------|-------------|---------|----------------------------
  1  | Rank Gauss      |    0.008    |   NO    | Needs full dataset for ranking
  2  | Log-Sigmoid     |    1.354    |   YES   | ← BEST ONLINE METHOD
  3  | Robust (IQR)    |    1.641    |   YES   | Needs median + IQR
  4  | Algebraic       |    2.141    |   YES   | No parameters needed
  5  | Tanh (Z-score)  |    2.310    |   YES   | Needs mean + std
  6  | Clipped 95%     |    2.975    |   YES   | Needs percentiles
  7  | Log-Normal      |    4.322    |   YES   | Just log transform
  8  | Box-Cox         |    8.393    |   NO    | Needs lambda estimation
  9  | Harmonic        |    8.730    |   YES   | Needs K parameter
 10  | Raw Data        |   68.020    |   -     | Baseline

Rank Gauss is the best overall (quality 0.008), but requires sorting the
entire dataset to compute percentile ranks → impossible during online training.
```

```
Log-Sigmoid is the best ONLINE-COMPATIBLE method (quality 1.354):
  - Only needs two pre-computed parameters per metric:  and
  - Parameters are stable across flow levels (log transform stabilizes)
  - Sigmoid bounds output to [0, 1] automatically
  - Monotonic: higher raw value → higher score → lower reward
```

5.5 Why Not Rank Gauss for Online RL?

Rank Gauss transforms data by: 1. Rank all values from smallest to largest 2. Map ranks to uniform [0, 1] 3. Apply inverse normal CDF (erfinv)

This requires the FULL dataset to compute ranks. In online RL, data arrives one sample at a time — no full dataset exists. Running Rank Gauss on partial data causes distribution drift as the dataset grows, destabilizing training.

VI. REWARD FUNCTION — A JUTALOM FÜGGVÉNY MATEMATIKÁJA

A LÉNYEG EGY MONDATBAN: A reward függvény a nyers forgalmi metrikákat (sec, mg/s) átalakítja egy 0 és 1 közötti számmá, ahol 0 = nagyon rossz forgalmi viszonyok, 1 = tökéletes forgalmi áramlás.

6.1 Log-Sigmoid Normalization Formula — LÉPÉSRŐL LÉPÉSRE

Legyen x a nyers metrika (pl. TotalWaitingTime = 120 sec):

1. LÉPÉS — Log transzformáció: $\log(x + \epsilon) \rightarrow \log(120 + 0.00001) \approx 4.787$

MIÉRT: A forgalmi adatok extrém jobbra ferdültek. Egy 30,000 sec-es WaitingTime nélküle teljesen eltorzítaná a skálát. A log "összenyomja" a nagy értékeket.

2. LÉPÉS — Z-score normalizáció: z = (log(x) -) / = (4.787 - 4.5848) / 1.8249 ≈ 0.111

Ez megmondja, hogy az érték hány szórásra van az ÁTLAGTÓL. z ≈ 0 → átlagos forgalmi viszonyok z > 0 → átlag feletti (rosszabb) z < 0 → átlag alatti (jobb)

3. LÉPÉS — Sigmoid leképezés: score = 1 / (1 + exp(-z)) = 1 / (1 + exp(-0.111)) ≈ 0.528

A sigmoid bármilyen valós számot [0, 1] közé visz: z = - → score ≈ 0 z = 0 → score = 0.5 (az "átlag" pont) z = + → score ≈ 1

4. LÉPÉS — Invertálás: reward_component = 1 - score = 1 - 0.528 = 0.472

MIÉRT: Mert a magasabb WaitingTime ROSSZABB. De a reward-ban a magasabb = jobb. Tehát megfordítjuk: alacsony WaitingTime → magas reward.

ÖSSZEFOGLALVA: z = (log(x + ϵ) -) / ... z-score log térben score = 1 / (1 + exp(-z)) ... sigmoid → [0, 1]

```
  reward_component = 1 - score       ... invertálás: kevesebb = jobb

A PARAMÉTEREK:
   = 1e-5 (= 0.00001) - numerikus stabilitás, hogy log(0) ne legyen -
```

```
    = E[log(X)] - a log-transzformált metrika átlaga a kalibrációs adatból
    = Std[log(X)] - a log-transzformált metrika szórása
```

6.2 Calibration Parameters — A KONSTANSOK ÉS HONNAN JÖNNEK

Ezek a számok a PCA adatgyűjtésből származnak (527,747 minta). A kód: sumo_rl_environment.py, ~490. sor körül hardcode-olva.

```
TotalWaitingTime:
   (MU_WAIT)  = 4.5848   → ez E[log(WaitingTime)] = log(~98 sec)
   (STD_WAIT) = 1.8249   → a szórás log skálán
  Median: 120.20 sec | P5: 3.20 sec | P95: 1,340.40 sec

  PÉLDA ÉRTÉKEK:
    WaitingTime = 3 sec    → z = (1.10 - 4.58) / 1.82 = -1.91 → reward = 0.87
    WaitingTime = 120 sec  → z = (4.79 - 4.58) / 1.82 =  0.11 → reward = 0.47
    WaitingTime = 1340 sec → z = (7.20 - 4.58) / 1.82 =  1.44 → reward = 0.19

TotalCO2:
   (MU_CO2)   = 10.8706  → ez E[log(CO2)] = log(~52,000 mg/s)
   (STD_CO2)  = 0.9629   → szűkebb szórás mint a WaitingTime-é!
  Median: 70,129.35 mg/s | P5: 8,382.93 mg/s | P95: 173,544.70 mg/s

  PÉLDA ÉRTÉKEK:
    CO2 = 8,383 mg/s   → z = (9.03 - 10.87) / 0.96 = -1.91 → reward = 0.87
    CO2 = 70,129 mg/s  → z = (11.16 - 10.87) / 0.96 =  0.30 → reward = 0.43
    CO2 = 173,545 mg/s → z = (12.06 - 10.87) / 0.96 =  1.24 → reward = 0.22

ÉRDEKES: A P5 és P95 értékeknél mindkét metrikánál ~0.87 és ~0.19
körüli reward-ot kapunk. Ez szándékos - a sigmoid pont így működik
2-nyi tartományban.
```

6.3 Combined Reward — A KÉT METRIKA ÖSSZEVONÁSA

reward = (w_wait × r_wait + w_co2 × r_co2) / (w_wait + w_co2)

Default súlyok: w_wait = 1.0, w_co2 = 1.0 → reward = (r_wait + r_co2) / 2 = EGYSZERŰ ÁTLAG

FONTOS: Osztunk a súlyok összegével (w_wait + w_co2)! Enélkül w=1.0 és w=1.0 mellett a reward [0, 2] lenne, ami megtévesztő. Az osztással MINDIG [0, 1] marad.

MIÉRT SÚLYOZOTT ÁTLAG ÉS NEM VALAMI BONYOLULTABB?

- Az egyszerű átlag jól kezeli a két metrika korreláltságát
- A súlyok (w_wait, w_co2) konfigurálhatók a training_config.yaml-ból
- Ha valaki jobban súlyozni akarja a CO2-t (pl. környezeti szemlélet): w_wait=1.0, w_co2=2.0 → a CO2 kétszer annyit számít

OUTPUT TARTOMÁNY [0, 1]: 0.0 = legrosszabb (extrém dugó + magas szennyezés) 0.5 = átlagos forgalmi viszonyok (a paraméterből adódóan) 1.0 = legjobb (nincs várakozás, minimális

kibocsátás)

SPECIÁLIS ESET — ÜRES CSOMÓPONT: Ha avg_waiting == 0 ÉS avg_co2 == 0 → reward = 0.0 (semleges) Miért nem 1.0 (= "tökéletes")? Mert az üres csomópontnál az ágens NEM CSINÁLT semmi jót/rosszat. Ha max reward-ot kapna, megtanulná, hogy "legjobb, ha nincs forgalom" → nem az, amit akarunk.

6.4 Reward Function Properties

1. BOUNDED [0, 1]: sigmoid guarantees finite output **2. SMOOTH:** differentiable everywhere → stable gradients for NN **3. MONOTONIC:** worse traffic → lower reward (consistent learning signal) **4. ONLINE-COMPATIBLE:** only needs 4 pre-computed constants **5. FLOW-INVARIANT:** log transform + z-score normalizes across traffic levels

- At 100 veh/h: waiting ~3 sec → z ≈ -1.5 → reward ≈ 0.82
- At 900 veh/h: waiting ~1340 sec → z ≈ +1.5 → reward ≈ 0.18
- Sigmoid maps this range smoothly, without clipping or saturation

6.5 Measurement Units Verification — MÉRTÉKEGYSÉG-ELLENŐRZÉS

Ez a szekció azért fontos, mert ha a PCA kalibrációs paraméterek (MU, STD) ELTÉRŐ mértékegységben lettek kiszámítva, mint amit a reward függvény kap bemenetként, az EGÉSZ normalizáció hibás.

```
TotalWaitingTime:
  SUMO: traci.lane.getWaitingTime(lane) → sec (egy sávon, egy timestepben)
  Kód: accumulated_waiting_time += step_waiting (5× akkumulálva)
  Reward-ba: accumulated_waiting_time / steps_measured = sec/substep
  PCA-ban: UGYANEZ a számítás → MU = 4.5848 → log(~98 sec)

TotalCO2:
  SUMO: traci.lane.getCO2Emission(lane) → mg/s (egy sávon)
  Kód: accumulated_co2 += step_co2 (5× akkumulálva)
  Reward-ba: accumulated_co2 / steps_measured = mg/s/substep
  PCA-ban: UGYANEZ → MU = 10.8706 → log(~52,000 mg/s)

  !!! VIGYÁZAT - EGY CSAPDA A KÓDBAN !!!
  Van egy get_total_co2_metric() függvény, ami /1000-rel oszt
  (mg/s → g/s konverzió). Ezt NEM használjuk a reward-ban!
  A reward közvetlenül az accumulated_co2 / steps_measured értéket
  használja (mg/s). Ha valaki átírná g/s-re, a MU_CO2 = 10.87
  nem lenne érvényes (g/s-ben MU  4.0 lenne).

KONZISZTENCIA ÖSSZEFOGLALÁS:
  PCA adatgyűjtés: accumulated / steps_measured → log() → mean, std
  Reward függvény: accumulated / steps_measured → log() → (x-MU)/STD → sigmoid
  → PONTOSAN UGYANAZ az adatfolyam  → a kalibrációs paraméterek érvényesek
```

VII. NEURAL NETWORK ARCHITECTURE — A HÁLÓ FELÉPÍTÉSE

EGY MONDATBAN: Minden RL ágens "agya" egy mély neurális hálózat, amely az observationből (fázis, timer, occupancy, flow) kiszámolja, hogy melyik akció (melyik fázis) mennyire jó.

7.1 Algorithm: QR-DQN (Quantile Regression DQN)

SIMA DQN (mit csinál):

- Egy neurális háló becsli: "Ha ebben az állapotban vagyok és az 'A' akciót választom, mennyi ÁTLAGOS jutalmat várhatok a jövőben?"
- Ez egyetlen szám (Q-value) minden akcióra.

QR-DQN (mit csinál jobban):

- Ahelyett, hogy egyetlen átlagot becsülne, az ELOSZLÁST becsli.
- 200 kvantilis (percentilis) értéket számol akcióként.
- Tehát tudja: "Az 'A' akció ÁLTALÁBAN jó (Q50=0.7), de van 20% esély, hogy nagyon rossz (Q20=0.2) — mert néha begyűrűzik a szomszéd dugója."
- **Ez a forgalomnál KRITIKUS:** ugyanaz a jelzésprogram néha jó, néha rossz, a véletlenszerű forgalmi mintáktól függően.

SB3 implementáció: sb3_contrib.QRDQN Default kvantilisek: N=200 (a return eloszlás 200 pontban becsülve)

7.2 Policy Network — A HÁLÓ STRUKTÚRÁJA

MultiInputPolicy (többes bemenetű policy):

BEMENET: Az observation Dict-et először "laposítjuk" (flatten):

- "phase": 1 szám (melyik fázis van)
- "phase_timer": 1 szám (hány ideje van ez a fázis)
- "occupancy": N szám (N detektor foglaltsága)
- "flow": N szám (N detektor átfolyása) → Összesen: 2 + 2N dimenziós vektor

Pl. ha 20 detektor van (tipikus): 2 + 2×20 = 42 bemeneti neuron.

REJTETT RÉTEGEK: [layer_size] × num_layers darab fully connected + ReLU réteg A layer_size és num_layers a SWEEP-ben keresett hiperparaméterek.

Példák: num_layers=2, layer_size=64 → [64, 64] → kis, gyors háló num_layers=3, layer_size=128 → [128, 128, 128] → közepes num_layers=5, layer_size=256 → [256, 256, 256, 256, 256] → nagy háló

A MÉRET SZÁMÍT:

- Túl kicsi háló: nem tud elég komplex stratégiát tanulni
- Túl nagy háló: lassabb, overfitting veszélye, több adat kell
- Ezért a sweep ezt is keresi!

KIMENET: num_actions × num_quantiles értékek Ha 4 fázis van és 200 kvantilis: $4 \times 200 = 800$ kimeneti neuron. Az ágens a LEGMAGASABB átlagos Q-értékű akciót választja (az exploration_fractionnal kontrollált ϵ-greedy stratégiával).

7.3 Model Saving — MENTÉS ÉS TRANSFER LEARNING

A modell két formátumban mentődik:

1. SB3 .zip (PyTorch natív):

- Tartalmazza a teljes modellt: háló súlyok, optimalizáló állapot, replay buffer, exploration rate, stb.
- **TRANSFER LEARNING-hez használható:** QRDQN.load("model.zip")
- Checkpoint-ok: 10k, 25k, 50k, 75k, 100k lépésenként
- Feltöltve WandB-re → bármikor visszatölthető

2. ONNX (Open Neural Network Exchange):

- Csak a háló struktúra + súlyok, futtatókörnyezet nélkül
- Deployment/inference céljára (gyors predikció)
- Bármilyen ONNX-kompatibilis runtime futtathatja (C++, Java, stb.)

MIÉRT FONTOS A ZIP MENTÉS: A sweep megtalálja a legjobb hiperparamétereket. Azzal a konfigurációval betanítjuk a modellt. A .zip modellt aztán TRANSFER LEARNING-gel finomhangolhatjuk más hálózatokra (pl. a valódi csomópont, amire alkalmazni akarjuk).

VIII. HYPERPARAMETER SWEEP — HOGYAN KERESSÜK A LEGJOBB BEÁLLÍTÁST?

EGY MONDATBAN: Nem tudjuk előre, milyen tanulási ráta, háló méret, stb. a legjobb. Ezért a WandB sweep automatikusan kipróbál sok kombinációt, és megkeresi azt, amelyik a legmagasabb átlagos reward-ot éri el.

8.1 Sweep Method: Bayesian Optimization

Konfig fájl: data/sweep_config.yaml Metódus: bayes (Bayesian optimalizáció) Cél: avg_reward MAXIMALIZÁLÁSA

HOGYAN MŰKÖDIK A BAYESIAN SWEEP? 1. Első néhány futás: véletlenszerű hiperparaméter kombináció 2. Építünk egy "surrogate model"-t: reward = f(learning_rate, batch_size, ...) 3. A surrogate model alapján kiválasztjuk, melyik kombináció ÍGÉRETES 4. Kipróbáljuk, frissítjük a modellt, ismételjük Ez SOKKAL hatékonyabb, mint a grid search (ahol MINDEN kombinációt kipróbálnánk — 4×4×100×40×6×4×4×4×4 = ~24 millió kombináció!).

A SWEEP AGENT-EK: Több párhuzamos "agent" futhat, mindegyik más hiperparamétert próbál. A start.sh megkérdezi, hányat akarsz párhuzamosan indítani. Az első agent libsumo-t használ (gyorsabb), a többi traci-t.

8.2 Swept Parameters — MIT KERESÜNK?

Paraméter | Típus | Tartomány | MIT JELENT?

```
----------------------|----------------|------------------|---------------------------
learning_rate         | log_uniform    | [5e-5, 5e-3]     | Milyen gyorsan tanul
batch_size            | categorical    | [32,64,128,256]  | Hány mintából tanul egyszerre
gamma (discount)      | uniform        | [0.90, 0.999]    | Mennyire fontos a JÖVŐ
```

```
exploration_fraction  | uniform        | [0.1, 0.5]       | Meddig kísérletezik
num_layers            | integer        | [1, 6]           | Háló mélysége
layer_size            | categorical    | [32,64,128,256]  | Réteg szélessége
buffer_size           | categorical    | [5k,10k,25k,50k] | Replay buffer mérete
flow_min              | categorical    | [50,100,150,200] | Min forgalmi szint
flow_max              | categorical    | [600,800,1k,1.2k]| Max forgalmi szint

PARAMÉTEREK RÉSZLETESEBBEN:

learning_rate (tanulási ráta):
  Milyen nagy "lépéseket" tesz a háló a tanulás során.
  Túl magas: instabil, a háló "ugrákat" tesz és nem konvergál.
  Túl alacsony: nagyon lassú tanulás, nem ér el jó eredményt időben.
  Log-uniform: azért logaritmikus, mert az 1e-4 és 1e-3 közötti
  különbség fontosabb, mint az 1e-3 és 9e-3 közötti.

gamma (diszkont faktor):
  Mennyire értékeli az ágens a JÖVŐBELI jutalmat a JELENLEGIVEL szemben.
  gamma = 0.9: "10 lépés múlva kapott jutalom = 0.9^10  0.35× mai érték"
  gamma = 0.99: "10 lépés múlva kapott jutalom = 0.99^10  0.90× mai érték"
  Magasabb gamma = "messzebbre lát" → jobb a forgalomszabályozáshoz,
  de nehezebb megtanulni (több variancia a Q-becslésben).

exploration_fraction:
  A total_timesteps hány százalékáig csökkenti az epsilon-t
  (a véletlenszerű cselekvések arányát).
  0.1 = gyorsan abbahagyja a felfedezést → ha a háló jó, gyorsan javul
  0.5 = sokáig próbálgat → ha sok lokális optimum van, jobb lefedés

num_layers + layer_size (HÁLÓ MÉRET):
  Ez a neurális háló kapacitását határozza meg.
  1 réteg × 32 neuron = 32 neuron → egyszerű, gyors, de korlátozott
  6 réteg × 256 neuron = 1536 neuron → komplex stratégiák, de lassú
  A SWEEP EZÉRT KERESI MINDKETTŐT - nem tudjuk előre, milyen komplex
  stratégia kell 21 csomópont kezeléséhez.
```

8.3 Fixed Parameters — EZEKET NEM KERESSÜK

delta_time = 5 (SUMO sec / RL step) — fizikai korlát, nem hiperparaméter reward_weights = {waiting: 1.0, co2: 1.0} — a PCA alapján döntöttük el train_freq = 4 (minden 4. lépésben tanul) — SB3 default, stabil gradient_steps = 2 (2× gradiens frissítés / tanítás) — alacsony overhead es_enabled = true — early stopping bekapcsolva es_patience = 20 epizód — ha 20 epizódig nem javul, leáll es_min_delta = 0.01 — minimális javulás küszöbe

8.4 Per-Episode Randomization — MINDEN EPIZÓDBAN MÁS

Ezek NEM hiperparaméterek, hanem RANDOMIZÁLT értékek. Minden epizód elején újra véletlenszerűen választjuk:

traffic_duration: random from [900, 1800, 2700, 3600, 5400, 7200] sec

```
flow_per_lane:     random uniform from [flow_min, flow_max] veh/h

Ez biztosítja, hogy az ágens NE tanuljon rá egyetlen forgalmi mintára.
```

IX. SIMULATION BACKEND — LIBSUMO vs TRACI

A LÉNYEG: Kétféleképpen lehet a SUMO szimulátorral kommunikálni. A libsumo gyorsabb, de csak egy példányban futhat. A traci lassabb, de több példányt tud egyszerre.

9.1 Two SUMO Connection Methods

traci (Traffic Control Interface):

- Kliens-szerver architektúra TCP socket-en keresztül
- A Python kód "üzeneteket küld" a SUMO-nak, ami egy külön processben fut
- ~30-50% overhead a szerializáció miatt
- Támogat TÖBB egyidejű SUMO példányt (más-más porton)
- GUI módhoz KÖTELEZŐ (a sumo-gui csak így működik)

libsumo (Library SUMO):

- Közvetlen C++ könyvtár hívások, nincs hálózati kommunikáció
- 30-50% GYORSABB mint a traci
- **SINGLETON:** csak EGY példány futhat processzenként! Ha két libsumo-t próbálsz indítani, összeakadnak.
- Nem használható sumo-gui-val

libtraci (Library TraCI):

- C++ natív kliens, teljes TraCI API kompatibilitás
- Gyors, mint a libsumo, de subscription-ök is működnek
- Nem singleton (több példány is futhat)
- macOS-en nincs PyPI csomag (csak ha a SUMO framework tartalmazza)

9.2 Selection Logic — AUTOMATIKUS VÁLASZTÁS

Import prioritás: libtraci > libsumo > traci

A USE_LIBSUMO környezeti változó irányítja: USE_LIBSUMO=1 → libsumo/libtraci (ha elérhető) USE_LIBSUMO=0 → mindig traci (párhuzamos futáshoz) nincs beállítva → libtraci próba → libsumo próba → traci fallback

PÁRHUZAMOS TANÍTÁS (start.sh): Futás #1: USE_LIBSUMO=1 (a leggyorsabb) Futás #2+: USE_LIBSUMO=0 (traci, mert a libsumo singleton) → Az első futás ~30-50%-kal gyorsabb lesz, mint a többi.

DOCKER: USE_LIBSUMO=1 mindig (minden konténer külön process) GUI: mindig traci (a libsumo nem tud GUI-t) **BASELINE EVAL:** mindig traci (hogy ne ütközzön a futó traininggel)

macOS (Apple Silicon) megjegyzés: A PyPI-ról telepíthető libsumo/libtraci csomagok x86_64

binárisok, amelyek nem futnak arm64-en. Megoldás: a SUMO framework natív arm64 libsumo-ját kell symlinkkelni a venv site-packages-be: /Library/Frameworks/EclipseSUMO.framework/.../tools/libsumo/

A kód: sumo_rl_environment.py, ~60-114. sor

9.3 SUMO Subscriptions — BATCH DATA RETRIEVAL

Az egyes TraCI API hívások (pl. traci.lane.getWaitingTime()) egyenként küldik az üzeneteket. 21 junction × 7 sáv × 5 metrika = 735 hívás/step. 50k step → ~36M API hívás → extrém lassú.

Megoldás: subscription-ök. Egyszer feliratkozunk, és minden simulationStep() után batch-ben visszakapjuk az összes adatot: traci.inductionloop.subscribe(det_id, [0x10, 0x13]) traci.lane.subscribe(lane_id, [0x5a, 0x7a, 0x60, 0x10])

A subscription-ök működnek traci-val és libsumo-val is (az alap subscription-ök, amelyek nem igényelnek extra argumentumokat). A _setup_subscriptions() metódus egy próba-subscription-nel teszteli, hogy a használt SUMO könyvtár támogatja-e.

X. CORRELATION MATRIX — MIT MUTAT A KORRELÁCIÓS MÁTRIX?

EGY MONDATBAN: A korreláció megmutatja, hogy melyik metrikák "mozognak együtt". Ha két metrika korrelációja >0.8, szinte UGYANAZT mérik — elég az egyiket használni.

10.1 Strong Positive Correlations (>0.8) — EGYÜTT MOZGÓK

```
TotalWaitingTime  QueueLength:         r  0.95  ← szinte azonosak!
TotalWaitingTime  AvgOccupancy:        r  0.85  ← nagyon hasonlók
TotalWaitingTime  VehCount:            r  0.80
TotalCO2  VehCount:                    r  0.90

MIT JELENT: Ha a WaitingTime nő, a QueueLength is nő (r=0.95).
Ez logikus: több várakozás = hosszabb sor. De ha MINDKETTŐT
beraknánk a reward-ba, az nem adna többlet információt -
gyakorlatilag 2× WaitingTime-ot használnánk.
```

10.2 Negative Correlations — ELLENTÉTES MOZGÁS

```
AvgSpeed  TotalWaitingTime:            r  -0.50
AvgSpeed  QueueLength:                r  -0.45

MIT JELENT: Amikor nő a várakozás, csökken a sebesség. Logikus.
De a korreláció "csak" -0.5, nem -0.9 - tehát a sebesség
NEM teljesen fordítottja a várakozásnak (van benne más is).
```

10.3 Near-Zero Correlations — FÜGGETLEN METRIKÁK

```
AvgCO2  szinte minden:                |r| < 0.15

MIT JELENT: Az AvgCO2 (jármű szintű kibocsátás) NEM korrelál
semmivel. Nem nő a forgalommal, nem csökken a sebességgel.
Ez azért van, mert ~1575 mg/s/jármű MINDIG, függetlenül a
forgalomtól. KÖVETKEZMÉNY: teljesen haszontalan mint reward.
```

10.4 Interpretation for Reward Design — MIT JELENT EZ NEKÜNK?

KONKRÉT KONKLÚZIÓ:

- **WaitingTime, QueueLength, VehCount, Occupancy:** REDUNDÁNSAK → Elég EGY belőlük. Mi a WaitingTime-ot választjuk (legmagasabb PC1).
- TotalCO2: korrelál a torlódással (r≈0.8), de NEM tökéletesen. A stop-and-go forgalom (gyorsulás-fékezés) TÖBB CO2-t termel, mint az egyenletes lassú haladás, UGYANAKKORA WaitingTime mellett. → **Hozzáad EGY EXTRA DIMENZIÓT:** a sebességprofilt. → Ezért van a reward-ban is.
- AvgCO2: r≈0 mindennel → SOHA nem használjuk.

XI. DATA STATISTICS — REFERENCE VALUES

11.1 Per-Metric Distributions (527,747 samples, flow 100-1200 veh/h/lane)

```
TotalWaitingTime [sec/step]:
  Mean:    367.82          Std:     1,100.94
  Median:  120.20          IQR:     323.60
  P5:      3.20            P95:     1,340.40
  Min:     0.20            Max:     31,348.00

AvgWaitingTime [sec/vehicle]:
  Mean:    5.50            Std:     11.45
  Median:  2.89            IQR:     5.12
  P5:      0.30            P95:     16.87
  Min:     0.003           Max:     287.60

TotalCO2 [mg/s/step]:
  Mean:    74,066           Std:     51,360
  Median:  70,129           IQR:     76,659
  P5:      8,383            P95:     173,545
  Min:     6.39             Max:     246,813

AvgCO2 [mg/s/vehicle]:
  Mean:    1,674            Std:     263
  Median:  1,576            IQR:     198
  P5:      1,468            P95:     2,262
  → Almost constant! Coefficient of variation = 15.7%
```

```
  → Useless as reward metric (no discriminative power)

TotalTravelTime [sec/step] (filtered, <1000s):
  Mean:    725.25          Std:     571.25
  Median:  594.13          IQR:     728.77
  P5:      79.31           P95:     1,830.34

TotalTravelTime_Raw [sec/step] (unfiltered, includes sentinels):
  Mean:    877,339         Std:     2,175,933
  Median:  6,063           P5:      145.49
  P95:     4,800,842       Max:     52,307,835
  → Extremely right-skewed due to sentinel values

VehCount [vehicles]:
  Mean:    44.4            Std:     ~35
  Median:  44.4            IQR:     52.1

AvgSpeed [m/s]:
  Mean:    ~2.3            Std:     ~2.5
  Median:  1.59            IQR:     2.62
  P5:      0.38            P95:     7.54

AvgOccupancy [%]:
  Mean:    ~45             Std:     ~30
  Median:  57.23           IQR:     37.41

QueueLength [vehicles]:
  Mean:    ~23             Std:     ~25
  Median:  18.00           IQR:     27.80
```

XII. LOG-SIGMOID NORMALIZATION PARAMETERS — ALL METRICS

Formula: z = (log(x + eps) - MU) / STD → sigmoid(z) → reward = 1 - sigmoid

Metric | MU | STD | Median | P5 | P95

Metric	MU	STD	Median	P5	P95
TotalTravelTime	6.2194	0.9644	594.13	79.31	1,830.34
AvgTravelTime	4.5295	0.8580	113.26	16.72	279.93
TotalTravelTime_Raw	9.8353	3.6341	6,063.10	145.49	4,800,842.0
AvgTravelTime_Raw	7.9540	3.5175	862.36	27.50	609,354.2
TotalWaitingTime	4.5848	1.8249	120.20	3.20	1,340.40
AvgWaitingTime	0.9783	1.2590	2.89	0.30	16.87
TotalCO2	10.8706	0.9629	70,129.35	8,382.93	173,544.70
AvgCO2	7.4120	0.1457	1,575.70	1,467.66	2,262.16
VehCount	3.4558	1.0533	44.40	4.00	113.80
AvgSpeed	0.4792	1.0377	1.59	0.38	7.54
AvgOccupancy	3.6915	0.9950	57.23	3.75	88.95

```
QueueLength            |  2.5926    |  1.2683    |        18.00 |         1.00 |        71.40

USED IN REWARD FUNCTION:
  MU_WAIT   = 4.5848   STD_WAIT = 1.8249
  MU_CO2    = 10.8706  STD_CO2  = 0.9629
```

XIII. ROBUST IQR NORMALIZATION PARAMETERS (alternative method)

Formula: tanh((x - median) / IQR) → reward = -tanh(...)

Metric | Median | IQR

```
----------------------|-------------|-------------
TotalTravelTime       |      594.13 |      728.77
AvgTravelTime         |      113.26 |      116.78
TotalWaitingTime      |      120.20 |      323.60
AvgWaitingTime        |        2.89 |        5.12
TotalCO2              |   70,129.35 |   76,658.74
AvgCO2                |    1,575.70 |      198.41
VehCount              |       44.40 |       52.10
AvgSpeed              |        1.59 |        2.62
AvgOccupancy          |       57.23 |       37.41
QueueLength           |       18.00 |       27.80
```

XIV. GENERATED ANALYSIS CHARTS — DETAILED INTERPRETATION

All charts in: metric_pca_test_v2/

==========
PCA CHARTS ==========

pca_01_scree.png — SCREE PLOT (Explained Variance) ————————————
MIT LÁTUNK: Oszlopdiagram, ahol minden oszlop egy főkomponens (PC1-PC12). Az Y tengely: az adott PC által megmagyarázott variancia százalékban. Kumulatív vonal is van rajta (piros).

HOGYAN OLVASSUK:

- **PC1 = 44.8%:** ez az első "dimenzió" a 12 metrikából. Önmagában majdnem az adatok felét megmagyarázza.
- **PC2 = 22.6%:** a második független dimenzió.
- **PC3 = 12.3%:** harmadik dimenzió (de ez a sentinel noise!).
- **PC1+PC2 = 67.4%:** két dimenzió az adatok 2/3-át lefedi.
- A többi PC (<6% egyenként) elhanyagolható.

MIT JELENT A CIKKBEN: "Az ábra igazolja, hogy 12 forgalmi metrikánk lényegében 2-3

független információdimenziót tartalmaz. A maradék 9 komponens redundáns — a mérőszámok egymás lineáris kombinációi."

pca_02_correlation.png — CORRELATION MATRIX (12×12) ——————————————————— **MIT LÁTUNK:** 12×12-es hőtérkép, RdYlBu_r színezéssel (kék=negatív, piros=pozitív). A cellákban a Pearson korrelációs együttható (r, -1 és +1 között). Szöveg színe kontrasztos: sötét háttéren fehér, világoson fekete.

HOGYAN OLVASSUK:

- Átló: mindig 1.00 (önmagával korreláció)
- **Bal felső 6×6 blokk (WaitingTime, Queue, VehCount, CO2, Occ):** erősen piros, r > 0.75 → ezek a TORLÓDÁSI KLASZTER. Mind ugyanazt mérik, más szemszögből.
- **TotalTT és AvgTT:** egymással r≈0.95, de a torlódási klaszterrel gyenge korreláció → FÜGGETLEN dimenzió (PC2).
- Raw TT: sem a torlódási klaszterrel, sem a filtered TT-vel nem korrelál erősen → ZAJDIMENZIÓ (PC3).
- AvgCO2: szinte minden sorban alacsony r → NINCS INFORMÁCIÓTARTALMA.
- AvgSpeed: negatív korreláció a torlódási metrikákkal (r≈-0.5) → logikus: több dugó = kisebb sebesség.

MIT JELENT A CIKKBEN: "A korrelációs mátrix azt mutatja, hogy a 12 metrika nem tartalmaz 12 független információt. A torlódási metrikák (WaitingTime, QueueLength, VehCount, TotalCO2, Occupancy) korrelációja r>0.75, tehát egy az egyben helyettesíthetők. Az AvgCO2 szinte semmivel nem korrelál — a jármű szintű kibocsátás közel konstans alacsony városi sebességeknél."

pca_03_biplot.png — PCA BIPLOT (Loading Arrows + Confidence Ellipses) ——————————————————————————— **MIT LÁTUNK:** Két tengelyes scatter plot, ahol:

- X tengely = PC1 (44.8% — Congestion dimenzió)
- Y tengely = PC2 (22.6% — Travel Time dimenzió)
- Színes ellipszisek = flow level-ek (100-1200 veh/h/lane) Mindegyik ellipszis az adott forgalmi szint 2 konfidencia- tartománya a PCA térben.
- Nyilak (vektorok) = metrikák "iránya" a PCA térben. Hosszabb nyíl = erősebb hatás. Irány = melyik PC-re hat.

HOGYAN OLVASSUK: Ellipszisek:

- Zöldes (100-300 veh/h): balra alul → alacsony torlódás, alacsony TT
- Sárgás (500-700 veh/h): középen → közepes forgalom
- Pirosas (900-1200 veh/h): jobbra → magas torlódás
- Az ellipszisek NEM fedik le egymást teljesen → a PCA valóban szétválasztja a forgalmi szinteket.
- Ha az ellipszisek teljesen átfednék egymást, a PCA nem lenne hasznos (a metrikák nem tudnák megkülönböztetni a forgalmi szinteket).

Nyilak (loading vektorok):

- **TotalWaitingTime, QueueLength, VehCount:** jobbra mutatnak (PC1+) → a torlódás irányába. Egymás közelében → REDUNDÁNSAK.
- **TotalTravelTime, AvgTravelTime:** felfelé mutatnak (PC2+) → a TT dimenzióba. Független a torlódástól.
- AvgSpeed: balra-felfelé → anti-korrelál a torlódással, de van TT komponense is.

- AvgCO2: a NYÍL ALIG LÁTSZIK, mert közel van az origóhoz. Ez azt jelenti, hogy AvgCO2-nak szinte nulla hatása van mindkét főkomponensre → HASZONTALAN mint reward metrika.

- TotalCO2: jobbra mutat (PC1+), de rövidebb mint WaitingTime → korrelál a torlódással, de kisebb mértékben.

- **TotalTT_Raw, AvgTT_Raw:** más irányba mutatnak, mint a filtered verzióik → ezek a SENTINEL NOISE-t képviselik.

MIT JELENT A CIKKBEN: "A biplot vizuálisan igazolja, hogy a forgalmi metrikák két fő klasztert alkotnak: a torlódási metrikák (jobb irány) és a menetidő metrikák (felső irány). A nyilak közelsége mutatja a redundanciát — WaitingTime, QueueLength és VehCount gyakorlatilag ugyanazt mérik. Az AvgCO2 nyíl az origón ül, megerősítve hogy nem alkalmas reward metrikának."

pca_04_waitingtime_boxplot.png — WAITING TIME PER FLOW LEVEL ———————————————————————— **MIT LÁTUNK:** Boxplot, ahol minden box egy forgalmi szint (100-1200 veh/h/lane). Y tengely: TotalWaitingTime [sec/step].

HOGYAN OLVASSUK:

- 100-300 veh/h: medián közel 0, kevés outlier → szabad forgalom
- 400-600 veh/h: medián emelkedik, IQR szélesedik → telítődés kezdete
- 700-900 veh/h: medián 200-500 sec, nagy szórás → időszakos dugók
- 1000-1200 veh/h: medián 500+ sec, extreme outlierek → krónikus dugó

MIT JELENT A CIKKBEN: "A WaitingTime exponenciálisan nő a forgalmi terheléssel, nem lineárisan. Ez igazolja a log-transzformáció szükségességét a reward normalizációban — lineáris skálán a magas forgalmi szintek teljesen elnyomnák az alacsonyakat."

==========
NORMALIZATION CHARTS ==========

01_Normalization_Matrix.png — HISTOGRAM MATRIX (8 metrics × 10 methods) ———————————————————————- **MIT LÁTUNK:** 8 sor (metrikák) × 10 oszlop (normalizációs módszerek) hisztogram mátrix. Minden cella: az adott metrika eloszlása az adott módszerrel normalizálva. Ideális: szimmetrikus, harang alakú eloszlás (Gauss-szerű).

HOGYAN OLVASSUK:

- Raw Data oszlop (1.): erősen jobbra ferdült, extrém outlierekkel
- Rank Gauss oszlop: tökéletesen szimmetrikus harang → ez a legjobb, DE offline módszer (nem használható RL-ben online).
- Log-Sigmoid oszlop: majdnem szimmetrikus, enyhe ferdeség → legjobb online módszer.
- Box-Cox: néhány metrikánál hosszú jobb farok → instabil.
- Harmonic: erős csúcsosság 0 közelében → rossz differenciálás.

MIT JELENT A CIKKBEN: "A hisztogram mátrix vizuálisan mutatja a normalizációs módszerek hatását. A Raw Data erős jobbra-ferdeséget mutat minden metrikánál, amit a Log-Sigmoid transzformáció közel szimmetrikus eloszlássá alakít."

02_Normalization_Quality.png — QUALITY BARPLOT ——————————————————— **MIT LÁTUNK:** 2×4 grid, minden subplot egy metrika. Oszlopdiagram: minden oszlop egy normalizációs módszer. Y tengely: quality = |skewness| + |excess_kurtosis|. Alacsonyabb = jobb (közelebb a Gauss-hoz).

HOGYAN OLVASSUK:

- Rank Gauss: mindig a legalacsonyabb (≈0) → tökéletes Gauss

- Log-Sigmoid: következő legjobb → 1-2 közötti quality
- Raw Data: mindig a legmagasabb → 10-200+ quality (extremális)
- Ha egy metrikánál MINDEN módszer magas quality-t mutat, az a metrika inherensen nehezen normalizálható.

MIT JELENT A CIKKBEN: "A quality metrika (|ferdeség| + |excess kurtosis|) objektíven rangsorolja a módszereket. A Log-Sigmoid 1.354-es átlagos quality- értékkel a legjobb online-kompatibilis módszer, mindössze egy nagyságrenddel rosszabb a Rank Gauss offline módszernél (0.008)."

03_Traffic_Dependency.png — NORMALIZED VALUE vs FLOW LEVEL ——————————————————— **MIT LÁTUNK:** Scatter plotok: X = forgalmi szint (veh/h/lane), Y = normalizált érték. Minden metrika és módszer kombinációra egy subplot.

HOGYAN OLVASSUK:

- Ideális: az Y értékek EGYENLETESEN szétterjednek a [0,1] tartományban minden forgalmi szintnél, nem csomósodnak.
- Ha egy módszernél a pontok egy vékony sávba tömörülnek → rossz differenciálás (az ágens nem tud tanulni a reward különbségekből).
- Log-Sigmoid: jól szétterített, monoton növekvő trend → a reward valóban tükrözi a forgalmi terhelést.

MIT JELENT A CIKKBEN: "A forgalomfüggőségi scatter plotok igazolják, hogy a Log-Sigmoid normalizáció megőrzi a monoton kapcsolatot a forgalmi terhelés és a normalizált érték között, miközben a teljes [0,1] tartományt kihasználja."

04_Reward_Comparison.png — REWARD DISTRIBUTION PER FLOW LEVEL ——————————————————— **MIT LÁTUNK:** Boxplotok forgalmi szintenként, a végső reward értékkel. 12 box (100-1200 veh/h), Y tengely = reward [0-1].

HOGYAN OLVASSUK:

- 100 veh/h: magas medián (~0.8) → szabad forgalom → jó reward
- 600 veh/h: medián ~0.5 → közepes → a sigmoid "közepe"
- 1200 veh/h: alacsony medián (~0.2) → dugó → rossz reward
- A boxok IQR-je (szélessége) mutatja a reward varianciáját. Túl nagy variancia → zajos tanulás. Túl kicsi → nincs differenciálás.

MIT JELENT A CIKKBEN: "A reward disztribúció forgalmi szintenként igazolja, hogy a Log-Sigmoid normalizáció értelmes reward grádienst biztosít a teljes [100, 1200] veh/h/lane tartományban."

05_Ranking_Heatmap.png — METHOD × METRIC QUALITY SCORES ——————————————————— **MIT LÁTUNK:** Hőtérkép: sorok = normalizációs módszerek, oszlopok = metrikák. Cellák: quality score (|skewness| + |excess kurtosis|). Színek: **ZÖLD (<1):** kiváló normalizáció **PIROS (1-10):** elfogadható de nem tökéletes **LILA (>10):** rossz normalizáció, nagy torzítás

HOGYAN OLVASSUK:

- Rank Gauss sora: csupa zöld → minden metrikánál tökéletes
- Log-Sigmoid sora: túlnyomóan zöld, néhol sárgás → jó kompromisszum
- Raw Data sora: csupa lila/piros → a nyers adatok nem normalizálhatók
- Box-Cox sora: vegyes (néhol zöld, néhol lila) → instabil módszer
- Ha egy OSZLOP (metrika) minden sorra lila → az a metrika inherensen problémás (pl. Raw

TT a sentinel értékek miatt)

MIT JELENT A CIKKBEN: "A heatmap összefoglalja, hogy a Log-Sigmoid módszer az egyetlen, amely minden reward-jelölt metrikánál egyformán jó minőséget ad, miközben online (streaming) környezetben is alkalmazható."

==========
CORRELATION MATRIX OLVASÁSI TIPPEK (pca_02_correlation.png) ==========

A szöveges értékek a cellákban: +1.00 = tökéletes pozitív korreláció (együtt nőnek) -1.00 = tökéletes negatív korreláció (ellentétesen mozognak) 0.00 = nincs lineáris kapcsolat

Színkódolás (RdYlBu_r colormap): Sötétkék = -1.0 (erős negatív) Fehér/sárga = 0.0 (nincs korreláció) Sötétpiros = +1.0 (erős pozitív)

Amit keresünk:

- Piros blokkok: redundáns metrikák → egy is elég a reward-ba
- Kék cellák: ellentétes irányú metrikák → potenciálisan komplementerek
- Fehér sorok/oszlopok: független metrikák → más dimenziót mérnek

==========
BIPLOT TENGELYEK ÉRTELMEZÉSE (pca_03_biplot.png) ==========

X tengely (PC1 — Congestion): Jobbra = TÖBB torlódás (magasabb WaitingTime, QueueLength, VehCount) Balra = KEVESEBB torlódás (szabadabb forgalom) Ez az első és legfontosabb dimenzió (44.8%).

Y tengely (PC2 — Travel Time): Felfelé = HOSSZABB menetidő (magasabb TravelTime) Lefelé = RÖVIDEBB menetidő Ez a második dimenzió (22.6%), független a torlódástól.

FONTOS: a tengelyek NEM fizikai mennyiségek, hanem a PCA algoritmus által megtalált "optimális nézőpontok". PC1 az a szempont, ahonnan a legtöbb variancia látszik; PC2 a második legtöbb, PC1-re merőlegesen.

A nyilak (loading vektorok) megmutatják, hogy az eredeti metrikák hogyan "vetülnek" erre a két szempont-tengelyre.

XV. BASELINE EVALUATION — ÖSSZEHASONLÍTÓ STRATÉGIÁK

Az RL ágens teljesítményének értékeléséhez 4 baseline stratégiát hasonlítunk össze, amelyek közül 2 a SUMO saját vezérlését használja (eltérő döntési tér), 2 pedig PONTOSAN UGYANAZT az action space-t és transition logic-ot, mint az RL ágens (fair összehasonlítás).

Script: baseline_actuated.py Használat: python baseline_actuated.py –episodes 20 –wandb-project <project>

15.1 Stratégiák Összefoglalása

```
+----------------+-------------------+----------------------------------------+
| Stratégia      | Döntési tér       | Leírás                                 |
+----------------+-------------------+----------------------------------------+
| static         | SUMO fix ciklus   | A .net.xml-beli program, nincs adaptáció|
| actuated       | SUMO fix sorrend  | Ciklus nyújtás/rövidítés detektorokkal |
| random         | RL action space   | Random fázisválasztás (worst case)     |
```

```
| max_pressure   | RL action space   | Legtöbb várakozó sávot zöldíti        |
+----------------+-------------------+---------------------------------------+

A "random" és "max_pressure" stratégiák a SumoRLEnvironment-et használják,
tehát a transition logic (intergreen matrix, yellow/all-red fázisok),
a reward függvény, és az observation space mind 100%-ban azonos az RL-lel.
```

15.2 Static Baseline

A SUMO .net.xml-ben definiált fix ciklusidejű jelzésprogram. **Pl. R1C1_C:** 42s zöld (É-D) → 3s sárga → 42s zöld (K-Ny) → 3s sárga = 90s ciklus. Semmilyen adaptáció nincs — a forgalmi helyzettől függetlenül mindig ugyanaz a ciklus fut.

15.3 Actuated Baseline

A SUMO beépített actuated vezérlése. Az eredeti fix fázisokból készül runtime-ban, TraCI-n keresztül:

Green fázisok: minDur = max(5s, duration × 0.5) maxDur = max(duration × 2.0, 60s)

```
Yellow/Red:     minDur = maxDur = duration (FIX, nem módosítható)

Működés: a green fázis legalább minDur-ig tart. Ezután, ha a detektor
maxGap (default 3.1s) ideig nem érzékel járművet, a fázis véget ér.
Ha folyamatosan jönnek járművek, a fázis maxDur-ig nyúlhat.
A ciklus sorrendje FIX (nem választhat fázist, csak nyújthat/rövidíthet).

FONTOS: az actuated NEM ugyanazt a döntési teret használja, mint az RL!
Az RL tetszőleges fázist választhat bármikor, az actuated csak a fix
sorrendű ciklust nyújtogatja. Ezért ez NEM fair összehasonlítás az RL-lel,
de az iparban ez a legelterjedtebb adaptív módszer, ezért referenciának
hasznos.
```

15.4 Random Baseline

Az RL environment-et használja, de random fázist választ: action = random.randint(0, num_phases - 1)

Ez a "worst case" baseline — ha az RL agent nem veri meg a random-ot, az azt jelenti, hogy a tanulás teljesen sikertelen.

FONTOS: a random sem vált azonnal — ha átmenet (transition) van folyamatban, az intergreen matrix szerint lejátssza a yellow → all-red → red-yellow szekvenciát, pont úgy, mint az RL ágens. Ezért ez fair összehasonlítás.

15.5 Max-Pressure Baseline (irodalmi referencia)

A max-pressure a közlekedési RL irodalom standard baseline-ja. (Varaiya, 2013; Wei et al., 2019)

Működés: minden döntési lépésnél kiszámítja az összes lehetséges green fázis "nyomását" (pressure), és a legnagyobb nyomásút választja:

pressure(action) = halting_vehicles(lane) lane ∈ green_lanes(action)

Ahol:

- green_lanes(action) = azok a bemenő sávok, amelyek az adott fázisban zöldet kapnak (a state string + getControlledLinks alapján)
- halting_vehicles = traci.lane.getLastStepHaltingNumber() (megállt járművek száma a sávon)

A fázis → sáv hozzárendelés felépítése: 1. getControlledLinks(junction_id) → link lista 2. Minden logic_phase-hez a phase_registry-ből kiolvassuk a state stringet 3. A state string i. karaktere 'G' vagy 'g' → link[i] zöldet kap 4. link[i] → incoming lane (a link tuple első eleme)

Példa (R1C1_C, 4 fázis): Action 0 → state "GGGrrr" → link 0,1,2 zöld → e11_e_0, e11_s_0, ... sávok Action 1 → state "GrGrGr" → link 0,2,4 zöld → más sávkombináció stb.

A max-pressure UGYANAZT az environment-et (SumoRLEnvironment) használja, mint az RL, ezért a transition logic, reward számítás mind azonos. Ha az RL agent nem veri meg a max-pressure-t, nincs hozzáadott értéke a neurális hálónak.

15.6 WandB Integráció

A baseline script minden stratégiát külön WandB run-ként logol:

- Run neve: "baseline-{strategy}" (pl. "baseline-max_pressure")
- Tag-ek: ["baseline", "{strategy}"]
- Logolt metrikák: avg_reward, episode, episode_length, global_step, és junction-szintű reward-ok (reward/R1C1_C, reward/R1C2_C, ...)

Így a WandB dashboard-on az RL training run-ok és a baseline-ok UGYANAZON a grafikonon megjeleníthetők, az avg_reward közvetlenül összehasonlítható.

15.7 Futtatás

Mind a 4 stratégia, 20 epizód, WandB-be is: python baseline_actuated.py –episodes 20 –wandb-project sumo-rl-single-2

Csak a fair összehasonlítók (RL action space): python baseline_actuated.py –episodes 20 –strategies random,max_pressure

Specifikus forgalmi tartomány: python baseline_actuated.py –episodes 30 –flow-min 300 –flow-max 800

Párhuzamos futtatás a traininggel (traci-t használ, nem ütközik libsumo-val): python baseline_actuated.py –episodes 20 & python main_headless.py –config training_config.yaml &

Kimenet: eval_results/baseline_summary.csv — epizódonkénti eredmények eval_results/baseline_per_junction. — junction-szintű részletek eval_results/baseline_comparison.png — box plot + junction diff chart

XVI. TRAINING PIPELINE — GUI ÉS CLI HIPERPARAMÉTER ÁTADÁS

A rendszer két módot támogat a tanítás indítására: 1. GUI (rl_trainer.py) — grafikus felületen beállítható hiperparaméterek 2. CLI (main_headless.py) — parancssori argumentumokkal (Docker, Colab, sweep)

16.1 GUI → Subprocess Hiperparaméter Átadás

A GUI (Tkinter) a tanítást KÜLÖN SUBPROCESS-ként indítja (main_headless.py). **A hiperparamétereket CLI argumentumként adja át:**

GUI mező → CLI argument → main_headless.py feldolgozás ———————————————————————————— Learning Rate → –learning-rate → hp["learning_rate"] Batch Size → –batch-size → hp["batch_size"] Buffer Size → –buffer-size → hp["buffer_size"] Gamma → –gamma → hp["gamma"] Exploration Frac. → –exploration-fraction → hp["exploration_fraction"] Num Layers → –num-layers → hp["num_layers"] Layer Size → –layer-size → hp["layer_size"] W (waiting) → –w-waiting → reward_weights["waiting"] W (CO2) → –w-co2 → reward_weights["co2"]

Logika (main_headless.py): 1. YAML-ból betölti a default hiperparamétereket (training_config.yaml) 2. CLI argumentumok felülírják a YAML default-ot (cli_hp_map dict) 3. WandB sweep paraméterek felülírják a CLI-t (ha sweep fut)

Prioritás: sweep config > CLI args > YAML defaults

16.2 WandB Sweep Integráció

A sweep_runner.py (vagy WandB agent) a main_headless.py-t hívja. A sweep config (data/sweep_config.yaml) felülírja a wandb.config-on keresztül a hiperparamétereket.

Több párhuzamos sweep agent futtatható: # Ugyanaz a sweep ID, több terminál: wandb agent <USERNAME>/<PROJECT>/<SWEEP_ID> # Terminal 1 wandb agent <USERNAME>/<PROJECT>/< # Terminal 2 # Docker-ben: docker-compose run -d sumo-trainer wandb agent <SWEEP_ID> # x N

FONTOS: az első agent libsumo-t használhat (gyorsabb), de ha több agent fut PÁRHUZAMOSAN, mindegyiknek traci-t kell használnia (USE_LIBSUMO=0), mert a libsumo nem thread-safe.

16.3 Docker Pipeline

A Docker container (docker-compose.yml) nem indít automatikusan tanítást. A command: ["echo", "..."] csak a használati utasítást írja ki.

Futtatás: docker-compose run –rm sumo-trainer python main_headless.py –config training_config.yaml docker-compose run –rm sumo-trainer python baseline_actuated.py –episodes 20 docker-compose run -d sumo-trainer wandb agent <SWEEP_ID>

16.4 Episode Logging

Minden epizód végén a következő metrikák logolódnak (WandB + konzol):

- episode_count: hányadik epizód
- episode_steps: lépések száma az epizódban
- avg_reward: átlagos reward az epizódon belül
- epsilon: jelenlegi exploration ráta
- traffic_duration: az epizód forgalmi időtartama
- flow_range: forgalmi tartomány [min, max]

XVII. CURRICULUM LEARNING — FOKOZATOS FORGALOMNÖVELÉS

KÉRDÉS: Gyorsítja-e a tanulást, ha először könnyű (kevés) forgalmat adunk, és fokozatosan növeljük a nehézséget?

17.1 Potenciális Előnyök

- Gyorsabb korai konvergencia: kevés forgalommal az ágens hamar megtanulja az alapvető fázisváltási logikát
- Stabilabb gradiens: kevesebb "lehetetlen" szituáció az elején
- Egyre komplexebb stratégiák: a nehezebb szintek építenek az alapokra

17.2 Problémák a Megközelítéssel

- **CATASTROPHIC FORGETTING:** a replay buffer fixed méretű (buffer_size). Ha növeljük a forgalmat, a régi "könnyű" tapasztalatok kiszorulnak, és az ágens elveszti a generalizáló képességét.
- **EXPLORATION GAP:** ha az exploration phase (ϵ-greedy, exploration_fraction) alatt csak könnyű forgalmat lát, a nehéz szituációkra nem fedez fel jó stratégiákat. Ez különösen problémás magas exploration_fraction (0.85) mellett.
- **DQN-SPECIFIKUS PROBLÉMA:** a DQN/QR-DQN replay buffer-re épít. A curriculum megváltoztatja az adateloszlást, amit a buffer tartalmaz, így a Q-value becslések "elavulnak" az új nehézségi szinten.

17.3 A Jelenlegi Megoldás: Domain Randomization + Target+Spread

A generate_random_traffic() függvény MINDEN epizódban véletlenszerű forgalmat generál, de NEM fix tartományból. A forgalomgenerálás két szinten randomizál (target+spread logika):

1. target = random.choice([100, 200, 300, 400, 500, 600, 700, 800, 900]) → az epizód összforgalmának nagyságrendje (9 diszkrét szint) 2. spread = random.choice([50, 100, 150, 200, 250, 300]) → az irányok közötti aszimmetria mértéke (6 szint) 3. flow_range = (max(50, target-spread), min(1100, target+spread)) 4. Sávonként: flow = randint(flow_min, flow_max)

- Időtartam: random [900, 1800, 2700, 3600, 5400, 7200] sec

MIÉRT NEM FIX TARTOMÁNY? A korábbi fix Uniform(100, 900) per lane megoldás problémája: ha sok sáv van (~80-146), a nagy számok törvénye miatt az összforgalom mindig ~500 veh/h/lane körül áll be — az ágens nem lát igazán könnyű vagy nehéz epizódokat.

A target+spread logikával:

- target variál → az ágens lát könnyű ÉS nehéz epizódokat
- spread variál → egyenletes ÉS aszimmetrikus forgalom is előfordul (pl. észak-délről 800, kelet-nyugatról 200 — reggeli csúcs főúton)
- Abszolút min: 50, max: 1100 veh/h/lane (gridlock elkerülése)

MIÉRT NEM A SWEEP KERESI A FORGALMI SZINTET? A forgalmi tartomány NEM hiperparaméter — a sweep dolga a háló architektúráját (learning_rate, num_layers, stb.) optimalizálni. A forgalomnak minden epizódban változatosnak kell lennie, hogy az ágens általánosítani tudjon. Ezért a flow_min/flow_max kikerült a sweep_config.yaml-ból.

Ez JOBBAN MŰKÖDIK, mint a curriculum, mert: 1. Az ágens MINDIG lát könnyű ÉS nehéz forgalmat 2. A replay bufferben vegyes nehézségű tapasztalatok vannak 3. Nem kell a curriculum ütemezéssel bajlódni (schedule design) 4. Az aszimmetrikus forgalom is megjelenik (spread randomizálás)

MEGJEGYZÉS: Ha egy külső hívó (pl. baseline_actuated.py) explicit flow_range-t ad az options-ben, az felülírja a target+spread logikát.

17.4 Ha Mégis Kipróbálnánk Curriculum-ot

A legjobb kompromisszum: az exploration phase alatt széles tartomány (pl. flow_range=[50, 1200]), utána szűkíteni a nehezebb tartományra. De a domain randomization már biztosítja ezt a hatást automatikusan.

KONKLÚZIÓ: A jelenlegi domain randomization + target+spread megközelítés preferált a curriculum learning-gel szemben, mert robusztusabb a catastrophic forgetting ellen, és nem igényel manuális schedule tervezést.

XVIII. KEY CONCLUSIONS FOR THE PAPER

1. PCA-BASED METRIC SELECTION:

- 12 traffic metrics reduce to 3 meaningful dimensions (79.7%)
- PC1 (44.8%) = congestion cluster — 6 metrics carry the same information
- PC2 (22.6%) = travel time — independent but less controllable
- PC3 (12.3%) = sentinel noise — confirms TT filtering is mandatory
- Optimal reward: TotalWaitingTime + TotalCO2 (PC1, complementary aspects)

2. NORMALIZATION FOR ONLINE RL:

- Rank Gauss is theoretically best (quality 0.008) but requires full dataset
- Log-Sigmoid is the best online-compatible method (quality 1.354)
- Only needs 4 pre-computed constants (2 per metric)
- Calibrated on 527K+ representative samples

3. SENTINEL VALUE HANDLING:

- SUMO getTraveltime() returns 200K-5M for empty/jammed lanes
- Filtering ($0 < tt < 1000$) moves TT from PC3 (noise) to PC2 (signal)
- Raw TT is unusable in reward functions

4. AvgCO2 IS UNINFORMATIVE:

- Nearly constant (~1575 mg/s/vehicle) across all traffic conditions
- PCA loadings ≈ 0 on PC1 and PC2
- HBEFA emission at low urban speeds is dominated by idle+cruise
- TotalCO2 (= AvgCO2 $\times$ VehCount) IS useful because it scales with congestion

5. REWARD DESIGN:

- Bounded [0, 1] via sigmoid
- Weighted average: (w_wait $\times$ r_wait + w_co2 $\times$ r_co2) / (w_wait + w_co2)

- Flow-invariant through log-transform normalization
- Empty junction → 0.0 (neutral, no false positive reward)

6. TRAFFIC GENERATION:

- Lane-level control (not O-D based) for reproducible per-lane density
- Epizód-szintű target+spread randomizálás (NEM fix tartomány)
- target ∈ {100..900}, spread ∈ {50..300} → flow_range = [max(50, t-s), min(1100, t+s)]
- A forgalmi szint NEM hiperparaméter — kikerült a sweep-ből
- Episode duration rotation [15min - 2h] prevents temporal overfitting

7. SIMULATION ENGINE SELECTION:

- Import priority: libtraci > libsumo > traci
- libsumo/libtraci for speed (30-50% faster), traci for parallel/GUI
- Automatic selection based on environment (local/docker/parallel)
- SUMO subscriptions for batch data retrieval (735 calls/step → 0)

8. BASELINE EVALUATION:

- 4 strategies: static, actuated, random, max-pressure
- random and max_pressure use the SAME environment as RL (fair comparison)
- Max-pressure: choose phase with most halting vehicles on green lanes
- Max-pressure is the standard baseline in traffic RL literature
- All baselines compute identical log-sigmoid reward for direct comparison
- WandB integration: baselines appear on same dashboard as RL runs

9. TRAINING PIPELINE:

- GUI (Tkinter) → subprocess (main_headless.py) → CLI args → WandB
- Hyperparameter priority: sweep config > CLI args > YAML defaults
- Docker: nem auto-start, explicit parancsokkal futtatandó
- Parallel sweep agents: USE_LIBSUMO=0 (traci) kötelező

10. DOMAIN RANDOMIZATION OVER CURRICULUM:

- Domain randomization (target+spread per episode) preferred over curriculum learning
- Curriculum learning risks catastrophic forgetting in fixed-size replay buffer
- Exploration gap: easy-only exploration → poor strategies for hard scenarios
- Target+spread ensures both easy/hard episodes AND symmetric/asymmetric traffic
- Flow_min/flow_max removed from sweep — traffic diversity is automatic, not a hyperparameter

11. WARM-UP KIKAPCSOLVA:

- Korábban 50-150 sec random fázisokkal → dugó az ágens első lépésekor
- Memória nélküli ágens (nincs LSTM) nem tudja megkülönböztetni az "örökölt dugó" és "saját hiba" szituációkat → zajos tanulási jel
- A forgalom jitter miatt úgyis fokozatosan indul (depart ≈ 0 + jitter)

12. TRAINING LOOP PERFORMANCE PROFILING:

- Inline profiler 100 lépésenként kiírja: predict, env.step, train, buffer
- Fő bottleneck: env.step (SUMO szimuláció, 53-97% a forgalomtól függően)
- Második: train (14-35%) — 21 ágens gradient update
- **KRITIKUS BUG JAVÍTVA:** train_freq=4 és gradient_steps=1 a configban volt, de a kód MINDEN lépésben 2 gradient step-et csinált (8x több mint kellett)
- Javítás után: FPS 3 → 8-9 (3x gyorsulás)
- Predict idő nő a lépésszámmal (1.9ms → 16ms) — PyTorch memória fragmentáció

13. JUNCTION-SZINTŰ INDEPENDENS TRÉNING (ARCHITEKTURÁLIS VÁLTÁS):

- A multi-agent (21 junction egyszerre) megközelítés ELHAGYVA
- OK: a globális normalizáció nem működik junction-szinten (OSM MU_WAIT=2.45 vs MEGA MU_WAIT=4.58 → 2.1 eltérés log-skálán)
- ÚJ: minden junction-re külön RL ágens, külön reward konstansok
- A teljes hálózat továbbra is szimulálódik (realisztikus cross-traffic)
- Per-junction metrika kalibráció → junction_reward_params.json
- WandB: egy projekt, group=junction_id, külön sweep-ek
- Transfer learning: katalógus → OSM junction, reward rekalibráció
- Lásd: XX. fejezet a részletekért

XX. ARCHITEKTURÁLIS VÁLTÁS — JUNCTION-SZINTŰ INDEPENDENS TRÉNING

HÁTTÉR ÉS MOTIVÁCIÓ: Az eddigi megközelítés (multi-agent, 21 junction egyszerre) egy globális reward normalizációs készletet használt. A metric_collection_osm.py eredményei bizonyították, hogy a normalizációs konstansok HÁLÓZAT- ÉS JUNCTION-FÜGGŐEK. Egyetlen junction metrikái drasztikusan eltérhetnek a globális átlagtól (OSM: MU_WAIT=2.45 vs MEGA: MU_WAIT=4.58 → 2.1 eltérés log-skálán). Ez lehetetlenné teszi az egységes reward-al való érdemi tanulást.

KUTATÁSI CÉL: Átfogó módszertan kidolgozása arra, hogyan lehet a valóságban DEPLOYOLNI traffic RL megoldásokat tetszőleges kereszteződésekre. A kereszteződés-könyvtár (catalogue) megmarad a pre-training alapjaként, de az ágensek egyenként tanulnak, egyedi reward kalibrációval.

XX.1 AZ ÚJ PIPELINE ÁTTEKINTÉSE

A módszertan 4 fázisból áll:

FÁZIS 1: PER-JUNCTION METRIKA KALIBRÁCIÓ — Minden egyes junction-re (pl. R1C1_C, R2C3_C, ...) külön lefut a metric_collection teszt:

- Input: mega_catalogue_v2.net.xml (teljes hálózat)
- De CSAK az adott junction forgalmát és metrikáit gyűjtjük
- Flow szintek: 100, 200, 300, ..., 1100 veh/h/lane
- Epizódok szintenként: 3
- Kimenet per junction: MU_WAIT, STD_WAIT, MU_CO2, STD_CO2

- PCA opcionális de hasznos: ellenőrzi, hogy a metrikák jellege (mely PC-re loadolnak) megegyezik-e a globálissal

A gyűjtés a meglévő metric_collection_test.py és metric_collection_osm.py mintájára történik, de automatizálva mind a 21 junction-re.

OUTPUT: data/junction_reward_params.json Formátum: { "R1C1_C": { **"MU_WAIT":** 3.21, "STD_WAIT": 1.65, **"MU_CO2":** 10.12, "STD_CO2": 0.85 }, "R1C2_C": { ... }, ... }

Ez a fájl a "katalógus tudásbázisa" — minden junction-höz tartalmazza az optimális reward paramétereket.

FÁZIS 2: ÖSSZEHASONLÍTÁS A GLOBÁLIS ÉRTÉKEKKEL ——————————————- Minden junction paraméterei összehasonlítva a korábbi globális **értékekkel (MU_WAIT=4.5848, MU_CO2=10.8706):**

- Eltérés mértéke (abszolút és %-os)
- Reward szimulálás mindkét paraméter-készlettel
- Reward range (p10-p90) — mekkora a tanulható grádiens?
- Heatmap: junction × paraméter eltérés mátrix

ELVÁRT EREDMÉNY:

- A szélső junction-ök (R1C1, R4C5 — kevesebb szomszéd, kevesebb bejövő forgalom) lényegesen eltérhetnek a belső junction-öktől (R2C3 — sok szomszéd, magas cross-traffic)
- **A split junction (R4C2_CE, R4C2_CW) különleges eset:** fél annyi sáv → fél annyi forgalom → alacsonyabb MU_WAIT

VALIDÁCIÓ:

- A reward függvény FORMÁJA (Log-Sigmoid) megmarad
- A metrika KIVÁLASZTÁS (TotalWaitingTime + TotalCO2) megmarad
- CSAK a 4 normalizációs konstans változik junction-önként

FÁZIS 3: EGYENKÉNTI JUNCTION TRAINING ——————————— **LÉNYEGI VÁLTOZÁS:** többé NEM egyszerre tanítunk 21 ágenst! Minden junction-re külön-külön fut a tréning, saját reward paraméterekkel.

3.1 SZIMULÁCIÓS BEÁLLÍTÁS:

- Hálózat: MARAD a mega_catalogue_v2.net.xml (nem vágjuk szét!)
- MIÉRT? A szomszédos junction-ök nélkül a forgalmi minta irreális: nem lennének eltorlaszolódó sorok, upstream visszahatás, stb. A teljes hálózat szimulálásával az ágens természetes forgalmi környezetet lát.
- RL ágens: CSAK az adott junction kapja az RL ágenst
- Többi junction: actuated control (SUMO beépített) vagy fix időzítéssel, hogy realisztikus legyen
- Forgalomgenerálás: az egész hálózatra generálunk forgalmat (target+spread), de a reward CSAK az adott junctionből számolódik
- Detektorok: csak az adott junction detektorai kellenek az observation-höz

3.2 IMPLEMENTÁCIÓ — sumo_rl_environment.py MÓDOSÍTÁSOK:

- **single_agent_id KITERJESZTÉSE:** nem csak OSM junction-re, hanem bármely mega_catalogue junction-re is használható
- ÚJ: junction_reward_params.json betöltése __init__-ben
- A reward konstansok kiválasztása az aktuális junction alapján: params = self.junction_params.get(junction_

global_defaults) MU_WAIT = params['MU_WAIT'] STD_WAIT = params['STD_WAIT'] MU_CO2 = params['MU_CO2'] STD_CO2 = params['STD_CO2']

- A többi junction-re NEM kell RL — static/actuated control

3.3 WANDB STRUKTÚRA: PROJEKT STRUKTÚRA (egy WandB projekt, csoportosítva): Projekt neve: pl. "sumo-rl-catalogue"

CSOPORTOK (wandb.init group= paraméter):

- "R1C1_C" — R1C1_C junction összes futása
- "R1C2_C" — R1C2_C junction összes futása
- ...

TAGS:

- junction neve (pl. "R1C1_C")
- "sweep" vagy "manual"
- "pre-training" vagy "fine-tuning"

SWEEP SETUP: Minden junction-höz KÜLÖN sweep futtatható: wandb sweep data/sweep_config.yaml \{} –project sumo-rl-catalogue \{} –name "sweep-R1C1_C" A sweep_runner.py paraméterként kapja a junction ID-t: python sweep_runner.py –junction R1C1_C A sweep az avg_reward-ot optimalizálja, de a reward az adott junction SAJÁT paramétereivel számolódik.

DASHBOARD:

- Filter by group → egy junction összes run-ja
- Runs table: junction oszlop a gyors szűréshez
- Panel: avg_reward, episode_length, epsilon, FPS
- Az avg_reward értékek junction-ök KÖZÖTT NEM összehasonlíthatók közvetlenül (más normalizáció!), de junction-ÖN BELÜL igen

FONTOS: A különböző junction-ök más-más optimális reward-al **fognak bírni! Ez NEM hiba:**

- Egyszerű junction (2 fázis, alacsony forgalom) → reward 0.7-0.8
- Komplex junction (4 fázis, magas cross-traffic) → reward 0.5-0.6
- A sweep minden junction-re KÜLÖN keresi a legjobb hiperparamétereket

3.4 MODELL MENTÉS:

- **Minden junction modellje KÜLÖN mentődik:** runs/sumo-rl-catalogue/R1C1_C_<timestamp>/ +– model_R1C1_C.zip (SB3 model) +– model_R1C1_C.onnx (exportált) +– reward_params.json (a használt MU/STD értékek) +– training_config.yaml (hiperparaméterek)
- A reward_params.json MINDIG mentődik a modellel együtt, mert a modell CSAK a saját normalizációjával működik helyesen

3.5 CLI HASZNÁLAT: # Egy junction tréning python main_headless.py \{} –junction R1C1_C \{} –project sumo-rl-catalogue \{} –timesteps 50000

Sweep egy junction-re wandb sweep data/sweep_config.yaml –project sumo-rl-catalogue python sweep_runner.py –junction R1C1_C

Összes junction szekvenciálisan (shell script) for jid in R1C1_C R1C2_C R1C3_C ...; do python main_headless.py –junction $jid –timesteps 50000 done

3.6 BACKGROUND JUNCTIONS KEZELÉSE: A teljes hálózaton a NEM RL-es junction-ök jelzőlámpái: Opció A: SUMO actuated program (beépített, adaptív) → Reális háttérforgalmi mintákat ad → A SUMO traffic_lights.json tartalmazza az alap programot Op-

ció B: Random fázisváltás (mint a metric collection tesztben) → Gyorsabb, de kevésbé realisztikus **AJÁNLÁS:** Opció A (actuated), mert a tanult ágens végül ilyen környezetben fog működni

FÁZIS 4: TRANSFER LEARNING — VALÓS DEPLOYMENT ——————————————— **ÚJ MÓDSZERTANI LÉPÉS:** a katalógus-beli junction-re tanított modellt alkalmazni egy VALÓS (OSM) junction-re.

4.1 A FOLYAMAT: 1. METRIKA KALIBRÁCIÓ az OSM junction-re

- Lefut a metric_collection_osm.py (vagy analóg script)
- Kiszámítja az OSM junction MU_WAIT, STD_WAIT, MU_CO2, STD_CO2-t
- Menti: data/osm_reward_params.json

2. TOPOLÓGIAI MATCHING

- Az OSM junction obs_space méretét összehasonlítjuk a katalógus junction-ökkel
- **MATCH feltétel:** azonos phase count + azonos detektor szám
- Pl. OSM 4 fázis, 6 detektor → match R1C1_C (4 fázis, 6 det)

3. SÚLY TRANSZFER

- A matchelő katalógus junction modelljét töltjük be
- A policy háló súlyai változatlanul átkerülnek
- CSAK a reward normalizáció változik (OSM paraméterekre)

4. FINE-TUNING

- Kis learning rate-tel (1/10 az eredeti) tovább tréningel
- Az OSM junction SAJÁT reward paramétereivel
- Kevesebb timestep kell (az ágens már "tud" jelzőlámpát kezelni)
- WandB: tag = "fine-tuning", group = OSM junction ID

4.2 MIÉRT GYORSÍT A TRANSFER LEARNING? A katalógus pre-training megtanítja:

- A "jelzőlámpa kezelés alapjait" (ne villogj, tartsd a zöldet)
- A detektorfoglaltság és akció közötti kapcsolatot
- Az intergreen mátrix kezelését (átmeneti fázisok) **A fine-tuning CSAK az adott junction sajátosságait tanulja:**
- Konkrét forgalmi minták (aszimmetria, csúcsirányok)
- Konkrét fázis-kombináció optimuma
- Az adott junction méretéhez igazított reward grádiens

4.3 VALIDÁCIÓ:

- Pre-training nélküli tréning vs transfer learning: Összehasonlítjuk a konvergencia sebességet és végső reward-ot
- Különböző source junction-ök hatása: melyik katalógus junction ad jobb transzfert az adott OSM junction-re?

——————————————————————— XX.2 IMPLEMENTÁCIÓS CHECKLIST ——————————————————————

A program NEM tanítási részei (GUI, intergreen matrix, netedit, transition planner, stb.) VÁLTOZATLANOK MARADNAK.

IMPLEMENTÁLANDÓ MÓDOSÍTÁSOK:

☐ 1. metric_collection_per_junction.py Automatizált PCA + metrika gyűjtés mind a 21 junction-re, egyenként. Kimenet: junction_reward_params.json Alap: metric_collection_test.py + metric_collection_osm.py

☐ 2. data/junction_reward_params.json Per-junction normalizációs konstansok (MU/STD) Generálja: metric_collection_per_junction.py

☐ 3. sumo_rl_environment.py — reward konstans betöltés junction_reward_params.json betöltése ___init___-ben Per-junction MU/STD kiválasztás a reward számításnál A jelenlegi if single_agent → OSM logika általánosítva

☐ 4. sumo_rl_environment.py — background junction kezelés single_agent módban a többi junction actuated/static NEM kapnak RL ágenst, de a forgalomgenerálás rájuk is vonatkozik

☐ 5. rl_trainer.py — wandb group + junction tag wandb.init(group=junction_id, tags=[junction_id, "pre-training"]) Per-junction sweep támogatás

☐ 6. sweep_runner.py — –junction CLI arg A junction ID-t paraméterként kapja Továbbadja a trainer-nek mint single_agent_id

☐ 7. main_headless.py — –junction CLI arg Az eddigi –single-agent analógiájára, de a mega_cataloguera

☐ 8. transfer_learning.py — reward rekalibráció Fine-tuning előtt automatikusan kiszámítja az OSM junction reward paramétereit (ha még nincs meg) A pre-trained modell source junction-jét kiírja logba

☐ 9. start.sh — junction választó menü Junction lista megjelenítése, választás lehetősége "Összes junction" opció szekvenciális futtatáshoz

☐ 10. Dokumentáció + Colab + Docker frissítés Az új pipeline tükrözése minden platformon

——————————————————— XX.3 FONTOS MEGKÖTÉSEK ÉS TRADE-OFF-OK ———————————————————

1. A TELJES HÁLÓZAT SZIMULÁLÓDIK (nem vágjuk szét): PRO: Realisztikus cross-traffic, upstream visszahatás CON: Lassabb, mint egy egyetlen junction szimuláció **TRADE-OFF:** A sebesség elfogadható, mert az RL ágens predict csak 1 junction-re fut (nem 21-re), és a train is csak 1 modellre

2. JUNCTION-ÖK KÖZÖTTI REWARD NEM ÖSSZEHASONLÍTHATÓ: Más normalizáció → más reward skála. Ez rendben van, mert a cél NEM a junction-ök rangsorolása, hanem minden junction-re a SAJÁT optimum megtalálása. WandB-ben a group szűrés segít.

3. SWEEP IDÉNYESSÉG: 21 junction × N sweep run = sok GPU/CPU idő. Lehetséges optimalizáció: ha két junction hasonló topológiájú (pl. R1C1_C és R1C3_C mindkettő 4 fázis, 6 det), a sweep eredmények átvihetők (warm start).

4. A REWARD PARAMS STABIL, NEM KELL GYAKRAN FRISSÍTENI: A normalizációs konstansok a HÁLÓZAT TOPOLÓGIÁJÁTÓL függnek, nem a tréning állapotától. Ha a net file nem változik, a junction_reward_params.json is érvényes marad.

XIX. BIBLIOGRAPHY / REFERENCES

SUMO: Lopez, P.A. et al. (2018). "Microscopic Traffic Simulation using SUMO." IEEE Intelligent Transportation Systems Conference (ITSC).

QR-DQN: Dabney, W. et al. (2018). "Distributional Reinforcement Learning with Quantile Regression." AAAI Conference on Artificial Intelligence.

HBEFA: Handbook Emission Factors for Road Transport, Version 4.2. https://www.hbefa.net/

PCA: Jolliffe, I.T. (2002). "Principal Component Analysis." Springer.

Stable-Baselines3: **Raffin, A. et al. (2021). "Stable-Baselines3:** Reliable RL Implementations." JMLR, 22(268), 1-8.

Log-Sigmoid Normalization: Custom method developed for this work, combining log-transform (variance stabilization) with sigmoid (bounded output).

Max-Pressure (baseline): Varaiya, P. (2013). "Max pressure control of a network of signalized intersections." Transportation Research Part C, 36, 177-195.

Wei, H. et al. (2019). "PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network." KDD 2019.

END OF DOCUMENT